



M256 Unit 14

UNDERGRADUATE COMPUTING

Software development with Java



Pursuing software
development

Unit **14**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007.

Copyright © 2007 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1562 0

CONTENTS

1	Introduction	5
1.1	The aims of this unit	5
1.2	Studying this unit	6
1.3	The practical exercises in this unit	6
2	Logical architectures	7
2.1	What is an architecture?	7
2.2	Package diagrams	7
2.3	Tiered architectures	9
3	Design patterns	13
3.1	Guidance for software design	13
3.2	The Façade pattern	14
3.3	The Singleton pattern	17
3.4	The Publish–Subscribe pattern	25
4	Multi-user systems	28
4.1	Distributed systems	28
4.2	The Remote Clinic System	30
4.3	Concurrent systems	34
5	New ways of developing software	39
5.1	The Rational Unified Process	40
5.2	Rapid application development	41
5.3	Agile development	42
5.4	Open source development	44
6	Summary	47
	Glossary	48
	References	51
	Index	52

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this unit we introduce a diverse selection of interesting and important issues from software development that M256 has not yet covered, but which arise quite naturally from the Hospital System and which you will be certain to meet again if you pursue software development further. In so doing, various themes that have run throughout the course will be drawn together and consolidated, and this should help you begin to revise important topics.

In *Unit 13* you participated in the final stages of implementing the Hospital System: completing a user interface and attaching it to the core system, thus enabling users to interact with the system. This is the culmination of the software development process in which you have been engaged throughout M256, learning along the way about ideas and techniques fundamental to software development. We hope that you felt a sense of achievement in developing your skills and in completing this system.

There are many different kinds of software system, however, and as systems get more complex (with a wide range of different kinds of functionality) so does their development. The ideas you will meet in this unit typically become more relevant as the complexity of software increases.

In the final section of this unit we conclude by placing the skills and understanding that we hope you have gained from M256 in the wider context of software development, through discussing some relatively new ways of developing software.

1.1 The aims of this unit

In this unit we aim to introduce you to the following topics:

- ▶ logical architectures (Section 2);
- ▶ tiered architectures (Section 2);
- ▶ design patterns (Section 3);
- ▶ persistence (Sections 3 and 4);
- ▶ distributed systems (Section 4);
- ▶ concurrent systems (Section 4);
- ▶ RUP, RAD and agile development methods, and open source development (Section 5).

The world of software development

The world of software development is a mixture of concerns ranging from correctness and execution efficiency to the beauty and elegance of the architecture, design and internal structure of systems to the overall aesthetics, usability and humanity of systems and all the way to the organization of development and the manner of software production. ... That is, at one extreme are concerns about the minimal requirements to get the job done and at the other are concerns about the qualities of the software and the humanity of the process that produced it.

Quotation taken from the Hillside Group website. Hillside is an organisation founded by leading software developers to promote the sharing of good object-oriented software development practice.

1.2 Studying this unit

The unit contains a number of SAQs and paper-based exercises which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

Sections 2 and 5 are short reading-based sections.

Sections 3 and 4 are longer sections which each involve some short practical exercises. You may need to refer to the *NetBeans Guide* when attempting these.

1.3 The practical exercises in this unit

When you are doing the exercises in this unit you may occasionally need to delete a data file.

Such files have a `.data` extension; for example, the one for the Hospital System is named `Hospital.data`. To delete such a file you should:

- 1 locate the project folder which contains the data file you wish to delete;
- 2 open this project folder and delete the `.data` file as you would any other file.

2 Logical architectures

In this section you will be introduced to the concept of a logical architecture, and in particular to the idea of a tiered architecture. You will also review the idea of a component, which you first encountered in *Unit 5*.

2.1 What is an architecture?

The term **architecture** is used in different ways in software engineering, the common theme being concentration on the overall organisation of a system: viewing the system in some abstract, high-level way, ignoring smaller details. For example, the view might be of the different computer elements that the system uses – hardware elements such as different processors and so on. The term **computer architecture** is often used in this regard. Or the view might be of how the different computers involved in a system are connected: the system's **network architecture**.

What we will be concerned with is usually referred to as a system's **logical architecture**. This is a view of the system's software which is concerned with the organisation of the software classes into larger components. It concentrates on how the overall system is constructed from major software components, how the components are connected and how they work together. This view is termed 'logical' because it is not concerned with how the system is physically deployed, across perhaps separate physical computers.

SAQ 1

What is meant by an object-oriented software component?

ANSWER.....

An object-oriented software component consists of a number of classes (perhaps just one), grouped together in some way, forming a piece of software with a well-defined purpose, which can be combined with other pieces of software to construct a larger system.

This idea was discussed in *Unit 5*, Subsection 3.3.

2.2 Package diagrams

An early design task within a software development project is to outline the system's logical architecture. We did this for the Hospital System when we decided that the user interface and the core system would be separate components. Such a design decision can be illustrated in a UML **package diagram**, as in Figure 1.

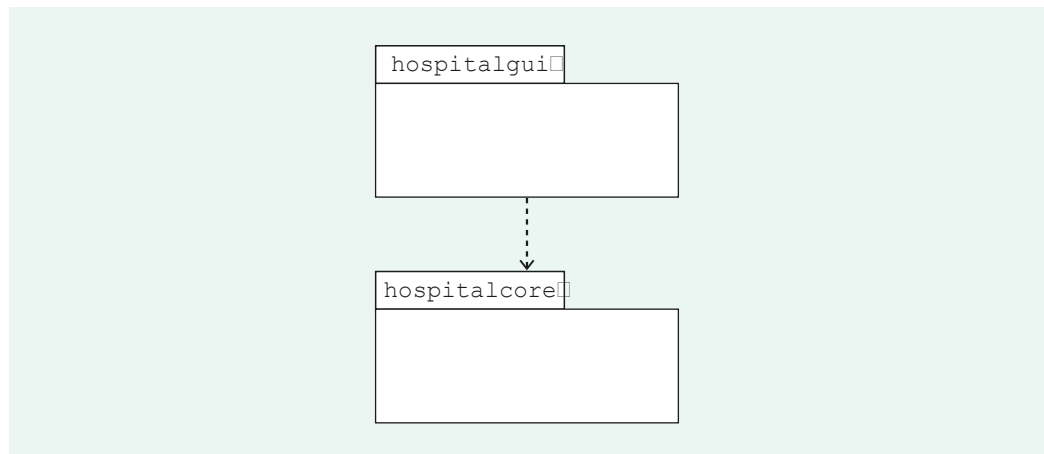


Figure 1 Two Hospital System components

In the figure, the dotted arrow signifies a **dependency** or **usage**: elements of the component `hospitalgui` use elements of the component `hospitalcore` in some way. In this case, objects in `hospitalgui` send messages to objects in `hospitalcore` (but not vice versa).

A package diagram may be employed, as here, to depict parts of a system that are eventually implemented as separate components (Java packages in the Hospital System). However, package diagrams can be used more generally to depict any grouping: the perspective depicted need not correspond to the eventual structure of the implemented system. For example, several teams of developers may work on the design of a particular component of a system. Even if this component is not decomposed into subcomponents, each team may visualise ‘their’ part of it and the parts of the component with which they are not directly involved as separate entities, i.e. as subcomponents.

SAQ 2

What might be the benefits to a team of visualising a component as separate subcomponents?

ANSWER.....

This enables team members to concentrate on the interaction between their part of the component and the rest of the component, abstracting away from the detail of how the rest works.

For simplicity, we will refer to the parts of a system illustrated in package diagrams as ‘components’ even though, as explained above, the intention may not be for each to be implemented as a separate component.

In large systems, package diagrams can be very useful for keeping track of the dependencies between different system components. As development progresses, additional components may be defined, and existing components refined into subcomponents.

Exercise 1

In the later stages of developing the Hospital System, an additional package, `m256people`, was defined. Both `hospitalgui` and `hospitalcore` depend on `m256people`. Illustrate the relationships between the packages `hospitalgui`, `hospitalcore` and `m256people` in a package diagram.

Discussion.....

Figure 2 shows our diagram.

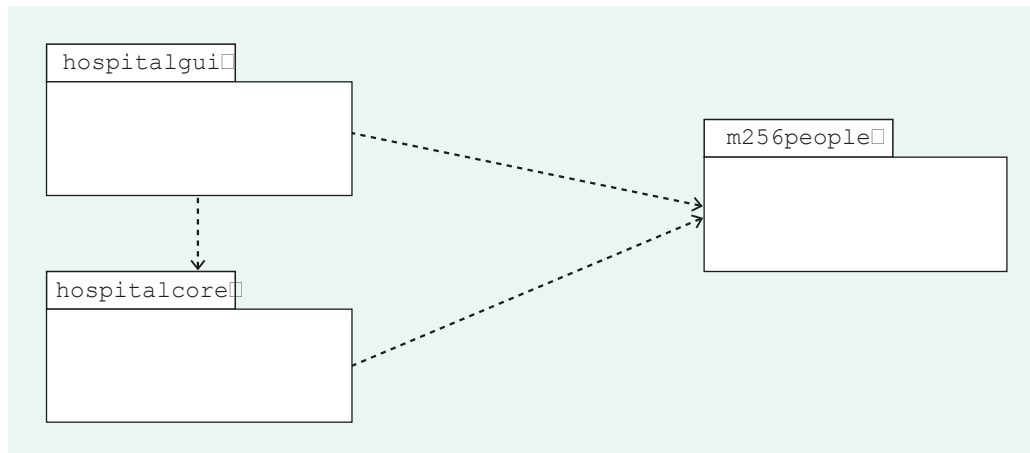


Figure 2 Three Hospital System components

2.3 Tiered architectures

Consider again the main components of the Hospital System: `hospitalgui` and `hospitalcore`, as illustrated in Figure 1. Partitioning the software design in this way led to the user interface and the core system being implemented as separate entities, in separate Java packages. The user interface may then be replaced without impact on the rest of the system, which is important since user interface requirements and technologies change rapidly. And the implementation of the core system may be changed, but as long as it provides the same services, the user interface will be unaffected.

Figure 1 shows that `hospitalgui` is dependent on `hospitalcore` but not the other way round. You may like to pause at this point and think about why it would be undesirable for the core system to be dependent on the user interface.

Clearly there has to be some communication between the two components, and it is obvious that the user interface needs to send messages to the core system. If the core system also sent messages to the user interface, then the two components would be more tightly coupled. This would make the job of replacing the user interface more difficult, since the developer would have to ensure that the new user interface could understand the same messages *from* the core system as did the old user interface, as well as sending the same messages *to* it.

Tiers

The user interface and the core system are conceptually very different things. The core system is a software representation of a part of the real world, whereas the user interface is just a way for a user to communicate with the core system. More complex systems may involve other kinds of conceptually different component; for example, many systems incorporate a database for persistent storage of data. A helpful strategy in managing increasing complexity is to structure a system's logical architecture as a **tiered architecture**, that is, as a series of **tiers** (or **layers**).

The layering we discuss is relevant for **information systems**, such as the Hospital System, which involve calculating and providing information for the user. For other kinds of system, e.g. embedded systems controlling devices like washing machines, such a structure may not be applicable.

Each tier can be considered as a component which has distinct, related responsibilities such that:

- ▶ lower-tier components have low-level, general responsibilities;
- ▶ higher-tier components have responsibilities that are increasingly specific to the particular business area;
- ▶ higher tiers may depend on lower tiers, but not vice versa.

For the Hospital System, Figure 1 depicts two tiers, a kind of architecture often referred to as a **client–server architecture**. More common is the classic **three-tiered** (or **three-layered**) **architecture**, illustrated in Figure 3, whose three tiers are traditionally described as follows.

- 1 The **user interface tier**, which gives a ‘view’ of the information provided by the system, using windows, spreadsheets, forms etc. In the context of a three-tiered architecture this tier is sometimes referred to as the **front end**.
- 2 The **business domain tier**, which corresponds to the core system and contains the processes of the system that are specific to the business area. These manipulate data and generate information; essentially, this is where the useful work of the system is carried out.
- 3 The **storage tier**, which provides a persistent data-storage mechanism, such as a database. This tier is the **back end**.

In the Hospital System, persistent storage is provided in a very simple way, using data files. Section 4 describes this in more detail.

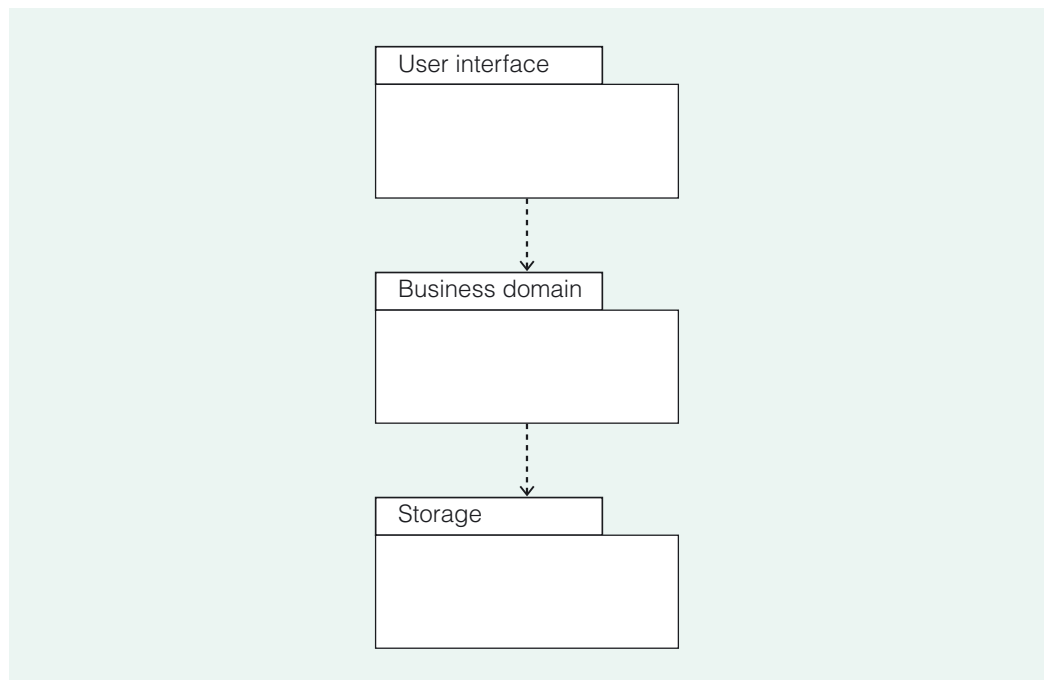


Figure 3 Classic three-tiered architecture

Note the following points about the three-tiered approach.

- 1 This *logical* depiction of the software as tiers implies nothing about the physical location of the elements involved. They may all run on the same computer, as with the Hospital System implementation that you met in *Unit 13*. Or they may be distributed, with several user interfaces running on different machines (e.g. the desktop computers of the hospital administrators), while the business domain resides on a separate computer and the data storage on perhaps yet another computer.
- 2 Structuring a system’s architecture on the basis of tiers as described above is only a general guideline, and variations are common. For example, Figure 2 shows both of the Hospital System tiers depending on `m256people`, and in fact different tiers often depend on common utility components.

Section 4 pursues the issue of physical location of software elements further.

- 3 The separation of the user interface and the business domain corresponds to the separation in the Hospital System of the user interface and the core system. It is in fact an application of what is known as the **model–view separation principle**. This states that not only should the business domain (the ‘model’) and the user interface (the ‘view’) be separate entities, but that the business domain should not depend on the user interface.

SAQ 3

State a disadvantage of the business domain depending on the user interface.

ANSWER.....

If the business domain depended on the user interface, then replacing the user interface would be more difficult, since any new user interface would have to ensure that it provided the same services to the business domain as the old one did.

The three-tiered architecture has proved very significant in software design, but it is limited. Suppose that the Hospital System requirements were changed to specify that several users should be enabled to use the system at the same time. It would be easy to create several user interfaces, but what if two users simultaneously tried to update the same aspect of the system? For example, they might each try to enter a patient into a ward with only one free bed. We are not interested here in the *details* of managing **concurrent access** to (i.e. simultaneous use of) the system but rather in what part of the software would most appropriately take on the management tasks required. Factoring this additional functionality into either the user interface or the core system seems intuitively inappropriate and would make them unnecessarily complex: managing concurrent access is not the business of either of them.

SAQ 4

What fundamental design principle is involved in the decision of where to locate responsibility for managing concurrent access?

ANSWER.....

The principle most obviously involved is that relating to cohesion. Assigning this responsibility to either the user interface or the core system would lower the cohesion of the chosen component.

In fact, an appropriate approach would be to incorporate an additional tier, between the user interface and the core system, to deal with concurrent access. This is illustrated in Figure 4.

Advantages of tiers

In general, as systems get more complex, tiered architectures are still relevant, but further tiers may be involved. This kind of logical architecture brings the following benefits.

- There is a separation of concerns between distinct parts of the system. One tier may be replaced by another that relies on the same services provided from the tier below and provides the same services to the tier above. The implementation of a tier may also be changed so long as the services it requires and provides are maintained.

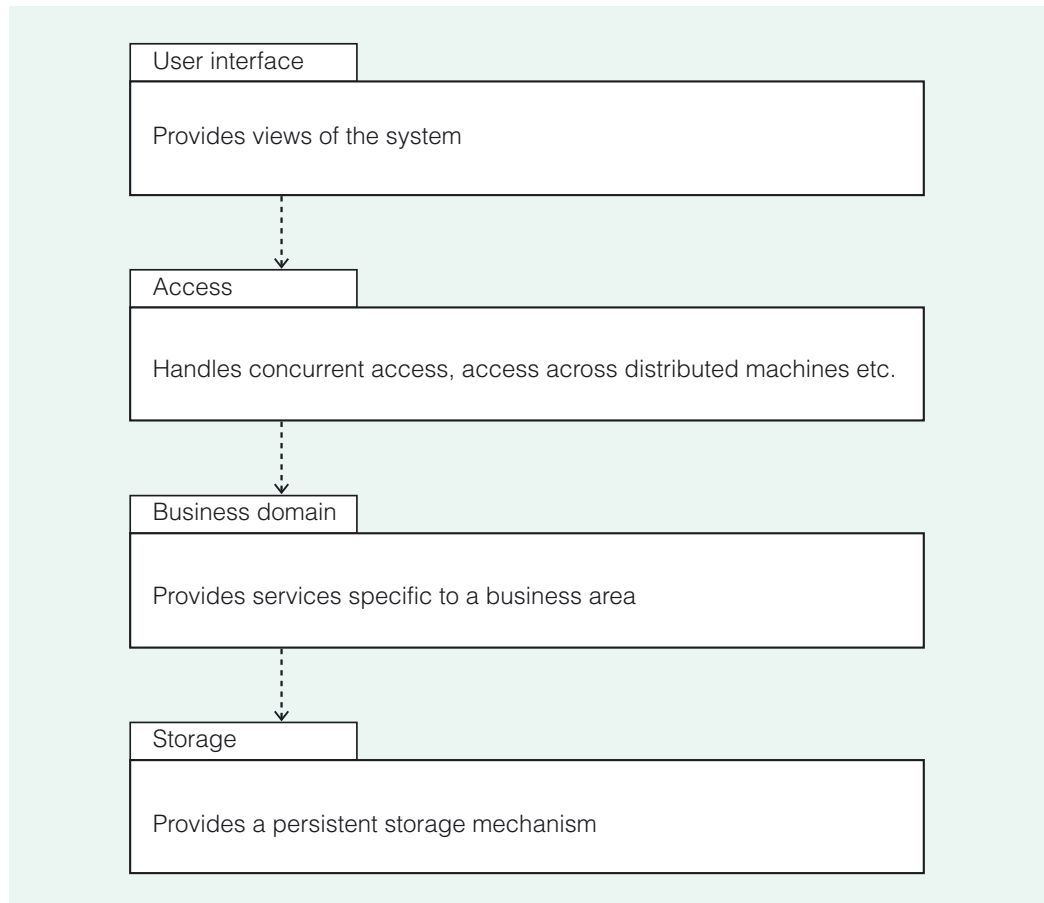


Figure 4 A four-tier system

- ▶ Interaction inside the system is streamlined and simplified.
- ▶ The system is easier to test and maintain.
- ▶ The development of the software is more straightforward, as different teams of developers can work on different tiers.
- ▶ Parts of the system – in particular the lower tiers – might be more easily reused in future systems. That is, design for reuse is facilitated.

SAQ 5

Can you think of another benefit of a tiered architecture?

ANSWER.....

Some tiers – in particular lower tiers such as databases – might be bought in or reused from previous projects. (In fact, databases of any significant complexity are almost always bought in.) That is, design *with* reuse is facilitated.

Design with reuse and design for reuse were discussed in *Unit 5*, Subsections 7.1 and 7.2.

In this section you have been introduced to the concept of a logical architecture, and to the idea of structuring a system's logical architecture into a series of tiers.

3

Design patterns

In this section you will meet the concept of design patterns, learning about their role in software development. You will then be introduced to some design patterns of particular relevance to the Hospital System.

3.1 Guidance for software design

Constructing a system in tiers, as you saw in Section 2, is a commonly used principle guiding the development of a system's logical architecture. In Subsection 2.2 of *Unit 7*, you met the concept of a software *design principle*, which guides the design of software at the more detailed level of interaction between individual objects.

You also met user interface design principles in *Units 11* and *12*. These are not the focus here.

SAQ 6

Briefly describe two software design principles.

ANSWER.....

Any two of the following, which were introduced in *Unit 7*, Subsection 2.2, would form a suitable answer here. (There are other design principles, which have not been considered in this course.)

- ▶ *Information expert*: assign a responsibility to the object that most readily has to hand the information needed to carry out the task.
- ▶ *Even distribution of responsibilities*: assign responsibilities evenly, with sensible delegation of tasks.
- ▶ *Cohesion*: assign responsibilities to maximise cohesion of individual classes, that is, to ensure that the responsibilities of each class are strongly related and focused in one area.
- ▶ *Coupling*: assign responsibilities to minimise coupling between classes, that is, to ensure that classes are not inappropriately dependent on each other.

Appropriate adherence to relevant design principles should lead to software which has desirable qualities.

SAQ 7

List two desirable qualities of software.

ANSWER.....

There are many desirable qualities. In M256, reusability and maintainability have been stressed. But others have been mentioned, for example (in *Unit 5*, Section 2), efficiency, portability and testability.

Design patterns, the subject of this section, are rather different from design principles, as you will see, but they provide another source of guidance in software design.

A **design pattern** is a tried and tested outline design applicable across a range of software systems, describing a recurring theme in elegant and respected designs.

There is some debate over precisely what constitutes a design pattern. Some developers would classify a design principle such as 'even distribution of responsibilities', suitably expressed, as a design pattern. Others reserve the term for more specific design ideas.

Typically, the description takes the form of a 'solution' which can be applied to a 'problem' which may arise in different contexts. It summarises the key elements of the 'problem' and 'solution' at an appropriate level of abstraction, omitting unnecessary detail and therefore lending it to being applied in a variety of concrete situations. Design patterns are in principle language-independent, though they are an object-oriented concept. Java in particular embodies many design patterns in its API packages and classes.

The application of a design pattern's solution in a particular situation should ensure that the software adheres to relevant design principles, and thus exhibits the desirable qualities mentioned above. Patterns, being concisely expressed and abstracting away irrelevant detail, enable developers to share good practice and to communicate more easily about complex systems. They particularly benefit novice developers, allowing them to reuse the expertise of others.

The Gang of Four

Design patterns are an evolving area of software engineering. They originated from ideas about designing buildings expressed by the architect Christopher Alexander, and gained popularity in computer science after the book *Design Patterns: Elements of Reusable Object-Oriented Software* was published in 1995, written by Gamma, Helm, Johnson and Vlissides – the so-called 'Gang of Four'. This book describes 23 patterns for object-oriented design and is considered a milestone in the history of object-oriented technology.

A common classification of design patterns is based on whether they are *structural* (concerned with the system's structure), *creational* (concerned with how objects are created), or *behavioural* (concerned with interactions within the system). We will look at one frequently applied example of each kind:

- ▶ the Façade pattern (structural);
- ▶ the Singleton pattern (creational);
- ▶ the Publish–Subscribe pattern (behavioural).

For each, a description of the pattern will be given and, to help you understand it, its relevance to the Hospital System will be explained.

3.2 The Façade pattern

We start with a design pattern whose application to simple systems like the Hospital System should by now be very familiar to you, even if its name is not. It is known as the **Façade pattern**, and its description is as follows.

Façade

Problem

A component of a system is made up of several classes, each contributing to the services provided by the component. Clients of the component potentially have to interact with all these classes to access the component's services. This makes using the component complex, and also means that changes to the internal workings of the component are likely to require changes to clients.

Solution

Add a new class – a **façade class** – to the component, to provide a primary means of interaction with the component. A client of the component does not

need to know about the internal workings of the component, because to access the component's services the client interacts with an object of this new class – a **façade object** – which does know about the structure of the component, and which organises the component's internal behaviour. The façade object provides a simple means of interacting with the component and avoids dependency of clients on the internal workings of the component. That is, it avoids excessive coupling.

Pictorially, the introduction of a façade class might be represented as in Figure 5.

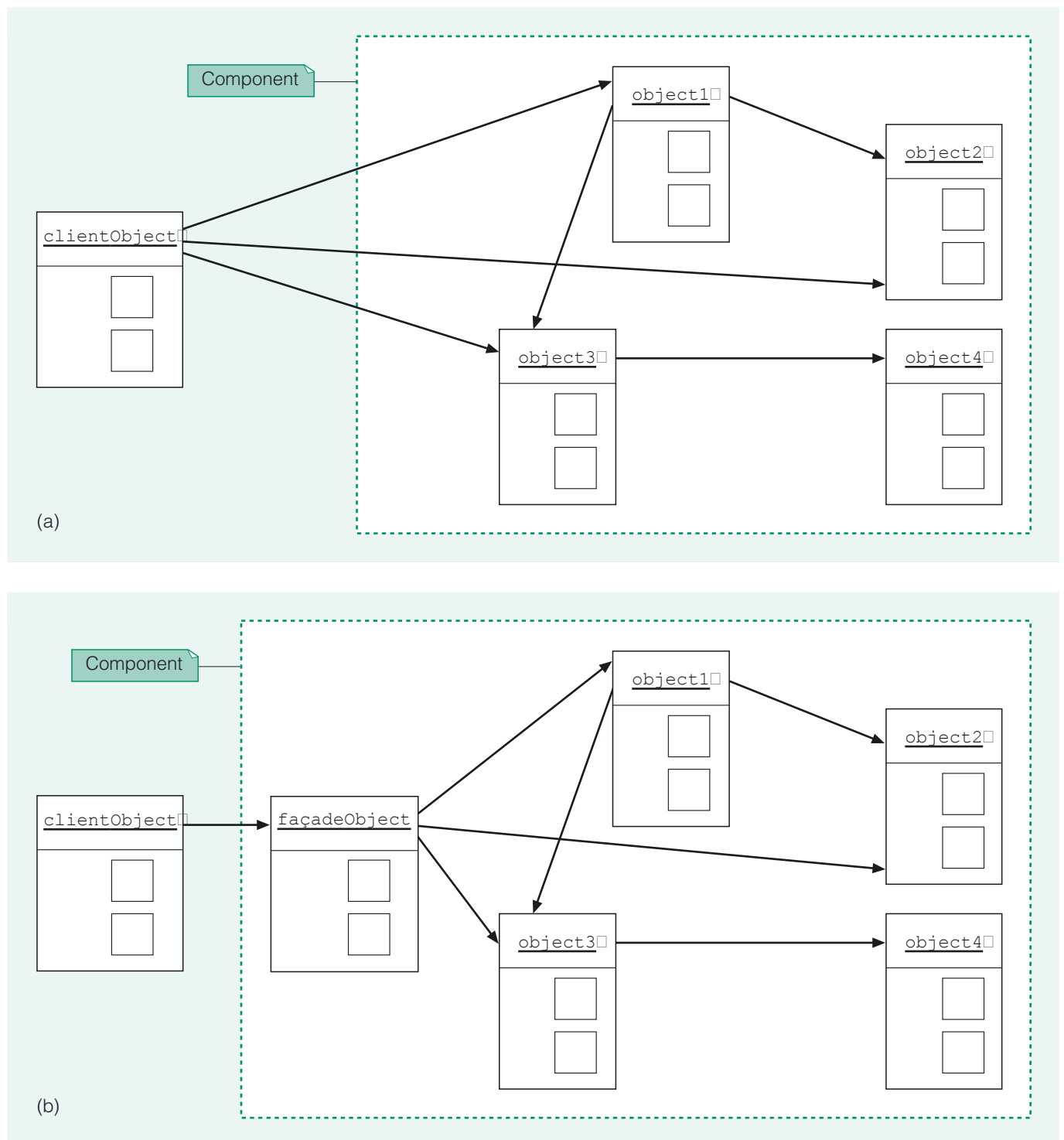


Figure 5 (a) Before the introduction of a façade class (b) After the introduction of a façade class

From the description of the Façade pattern, you should recognise that, although it has not previously been referred to by name, this classic pattern is embodied in M256's Hospital System.

SAQ 8

In developing the core system component of the Hospital System, what terms were used instead of *façade class* and *façade object*?

ANSWER.....

The terms *coordinating class* and *coordinating object* were used.

Façade classes are very common in object-oriented design. The Java API, for example, contains many classes concerned with drawing figures, but most programmers do not need to know about these. Instead they use the façade class `Graphics`, which offers the most commonly required services.

There are variants of the Façade pattern that involve the use of several façade classes instead of one, or that provide the client with limited direct access to some internal classes. The Façade pattern can also be used to unify a collection of legacy systems. We will look briefly at each of these possibilities.

Several façade classes

For a complex component offering distinct services, a variation on the Façade pattern is to add a façade class for each service, or each subset of services.

For example, when a core system implements many complex use cases, rather than there being one coordinating class whose coordinating object deals with all the use cases, there might be a coordinating class and a coordinating object for each use case: classes such as `AdmitPatientCoordinator`, `TreatPatientCoordinator` etc.

SAQ 9

In a system dealing with many complex use cases, what might be the benefits to the client (e.g. the user interface) of having a separate coordinating class for each use case?

ANSWER.....

Instead of having to contend with the complexities of a single coordinating class with many methods, a client would, for any given use case, only have to deal with the particular coordinating class that offers the functionality required.

These issues were discussed in *Unit 5*, Subsection 7.2.

At the heart of the issue raised in SAQ 9 is the cohesion of a coordinating class. If a single coordinating class handles many complex use cases, this class is likely to be 'bloated' with excessive responsibilities, that is, to have low cohesion.

SAQ 10

State two disadvantages of having a coordinating class with low cohesion, aside from the client's perception of complexity.

ANSWER.....

The class is likely to be difficult to test and difficult to maintain.

Direct access to internal classes

Variations on the Façade solution may permit clients limited access to the other classes inside the component in question, with the façade object remaining the main point of access for all state changing and complex information retrieval.

SAQ 11

Consider the Hospital System. Apart from access to the coordinating object (i.e. the façade object in the core system), what kinds of access do clients have to core system objects?

ANSWER.....

Clients can send only getter messages directly to the other core system objects.

Legacy systems

Another situation in which a façade class is useful is in unifying previously separate subsystems which are being combined into a larger system. The subsystems might be legacy systems, for example, which persist despite employing outdated technologies or languages. Instead of having to interact in different (and perhaps complicated or outdated) ways with different subsystems, a client interacts in a single unified manner with a façade object, which itself takes on the task of interacting with the subsystems, thus shielding the client from complexity.

Subsection 2.1 of *Unit 5* mentioned the concept of a legacy system.

3.3 The Singleton pattern

To motivate discussion of this pattern, you will first look at how objects of the class `HospCoord` are created in the Hospital System.

Creating coordinating objects

Recall that we have been referring to `HospCoord` as the *coordinating class*, and to an object of this class as a *coordinating object*. You may also recall that we mentioned, in early units of the course, the expectation that a coordinating object would be the *only* object of its class. However, you could be forgiven for overlooking this point until now, since it has not subsequently been considered why this uniqueness is appropriate or how it could be achieved.

Underlying the development of the Hospital System was the assumption that there is just one real-world hospital involved: the one described in the requirements document. Since the core system represents this hospital, there should only ever be one core system in existence. It would be confusing, and error-prone (as you will see in detail in Exercise 2), for there to be more than one core system representing the same hospital. However (as you will also see) each time a distinct coordinating object is created, so too is a distinct core system: quite naturally, since the purpose of the coordinating object is to be the 'public face' of its core system.

To prepare you for the next exercise, we will look at how a coordinating object for the Hospital System is created. Below are outline parts of code from the class `HospCoord` (from the implementation you met in *Unit 10*), relating to the creation of a new object of that class.

First, here is the constructor for the class.

```
private HospCoord()
{
    teams = new HashSet<Team>();
    wards = new HashSet<Ward>();
    // initialise the wards, teams and doctors using the default file
    readHospitalDetails("hospital.csv");
}
```

Note the following about this constructor.

- ▶ It is declared as `private`, so a client (such as a user interface) cannot use it to create a new `HospCoord` object.
- ▶ The code, which calls on the helper method `readHospitalDetails(setupFile)`, sets the instance variables `teams` and `wards` of a new `HospCoord` object to reference collections of new `Team` and `Ward` objects, and assigns new `Doctor` objects to the `Team` objects, taking the details for these objects (ward names for example) from a default file. That is, the constructor sets up a default core system, mirroring the establishment of a new hospital, with teams, wards and doctors (but no patients).
- ▶ If a client could invoke this constructor to create a `HospCoord` object, then each time the Hospital System was started and a coordinating object created, the core system would be in the same state – representing the default teams, doctors and wards but no patients. All details of patients admitted during the last execution of the system would be lost.

Second, here is a static (class) method of the class `HospCoord`.

```
public static HospCoord getHospital()
{
    HospCoord hospital = null;
    try
    {
        [code omitted: attempt to set up hospital with details from the file
        Hospital.data]
    }
    // if there is no file i.e. no previous details were saved
    catch(FileNotFoundException ex)
    {
        // invoke the constructor and thereby set up
        // hospital with the default initial state
        hospital = new HospCoord();
        ...
    }
    return hospital;
}
```

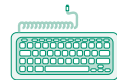
Note the following about this method.

- ▶ This static method is available for use by clients, since it is declared `public`. It creates and returns a `HospCoord` object. (Since it is a static method it can be invoked before any object of the class exists.)
- ▶ The code looks to see if previously saved details (i.e. details saved from a previous execution of the system) can be found in a file `Hospital.data`. If so, a new `HospCoord` object is set up using these details. That is, a core system is created, which – although it contains entirely new objects – represents the same hospital details as were saved by the previous execution.

- ▶ If no previously saved details are found, then a new `HospCoord` object, and its new core system, are set up using the default hospital details (by invoking the constructor).
- ▶ This method therefore enables details of patients to persist between consecutive executions of the system. Of course, this relies on there being some mechanism for saving the details to `Hospital.data` in the previous execution: this is the purpose of `HospCoord`'s `save()` method, which you do not need to consider here.

The implementation of `getHospital()` at present plays a useful role in enabling data to persist between consecutive executions. However, it is flawed. By repeatedly using `getHospital()`, a client (such as a user interface) can create any number of distinct `HospCoord` objects and thus any number of distinct core systems. In the following exercise you will discover why this is problematic.

Exercise 2



In NetBeans open the project `Two_coordinating_objects_Ex_2`, which is in the folder `M256\M256Code\Exercises\Unit14`.

The project consists of three packages, as follows.

- 1 `hospitalcore`. This is the Hospital core system, containing amongst other classes the coordinating class `HospCoord`.
- 2 `hospitalclient`. This package contains a very simple client of the Hospital core system.
- 3 `m256people`. This is a utility package used by both `hospitalcore` and `hospitalclient`.

The project contains no `HospGUI` package.

Read the code for the class `HospitalClient` in the package `hospitalclient`.

- (a) Part 1 of the code creates two `HospCoord` objects, by twice invoking the static method `getHospital()` on `HospCoord`, and checks that they are indeed different objects.

Their core systems should at this stage, however, represent the same real-world hospital: the code checks this by printing out details of each `HospCoord` object.

Run the project.

For brevity, we will refer to these objects as `hospital1` and `hospital2`, although of course `hospital1` and `hospital2` are actually references to the objects.

- (b) The purpose of Part 2 of the code is to print out the initial details of the `Patient` objects in the core systems of each of `hospital1` and `hospital2`. The purpose of Part 4 of the code, is to print out the final details of the `Patient` objects in each core system. (You will consider Part 3 in part (c).)

The `HospCoord` method `getPatients()` returns a collection of all the `Patient` objects. Use this method in adding the required lines of code in Parts 2 and 4, then run the project.

- (c) The purpose of Part 3 of the code is to admit a patient to the hospital. First, a name and a date of birth are created, and a `Team` object is identified.

Add the line of code required in Part 3 to admit the patient to the hospital represented by `hospital1`, using the `HospCoord` method `admit(aName, aSex, aDate, aTeam)`. (`Sex.F` is the required value for `aSex`.) Now run the project – what has been the effect on `hospital2` of admitting the patient?

- (d) Run the project again. What patient details are recorded from the previous execution of the system? Can you explain this?

Discussion.....

- (a) The output should confirm that `hospital1` and `hospital2` reference different objects. The client code creates these two distinct `HospCoord` objects, each with its own `Team`, `Ward` and `Doctor` objects. That is, the two core systems are distinct – they have no objects in common. However, they represent exactly the same real-world hospital details – the same wards, teams and doctors (though they may be listed in different orders in the output).

- (b) When you have added the code, Part 2 should look like this:

```
System.out.println("Initial patient details of hospital1:");
System.out.println(hospital1.getPatients());
System.out.println("Initial patient details of hospital2:");
System.out.println(hospital2.getPatients());
```

Part 4 should look like this:

```
System.out.println("Final patient details of hospital1:");
System.out.println(hospital1.getPatients());
System.out.println("Final patient details of hospital2:");
System.out.println(hospital2.getPatients());
```

When the project is run, the output should indicate that there are no `Patient` objects corresponding to either `hospital1` or `hospital2`.

- (c) The required line of code is:

```
hospital1.admit(theName, Sex.F, theBirthDate, theTeam);
```

When the project is run, the output should include the following:

```
Initial patient details of hospital1:
[]
Initial patient details of hospital2:
[]
Admitting a patient to hospital1...
Final patient details of hospital1:
[Ms Bet Lynch female 23/05/78]
Final patient details of hospital2:
[]
```

This output demonstrates that admitting a patient to the hospital represented by `hospital1` has no effect on the hospital represented by `hospital2` – a problem when these are supposed to be the same hospital! This is because each coordinating object has its own distinct core system, so changes to one have no impact on the other.

- (d) Running the project again reveals that no patient details were saved from the previous execution of the system. This can be seen from the following part of the output.

```
Initial patient details of hospital1:
[]
Initial patient details of hospital2:
[]
```

The effect noted here can be explained as follows.

At the end of the previous execution (the last time the system was run) the following lines of code (Part 5) were executed:

```
hospital1.save();
hospital2.save();
```

The first line saved `hospital1`'s core system details to the file `Hospital.data`, including those of the patient (Bet Lynch) admitted to the hospital in that execution.

However, the second line then overwrote these details by saving `hospital2`'s core system details to the same file, `Hospital.data`. These, as you saw in part (c), do not include details of the patient admitted via `hospital1`.

On running the project again two new instances of `HospCoord` – coordinating objects – were created. But each time a coordinating object is created its core system is in the same state as the last one that was saved to the file `Hospital.data` (or in the default state if no previous `HospCoord` object has been saved). Hence the changes made to `hospital1` were lost.

The project `Two_coordinating_objects_Ex_2_Sol`, which is in the folder `M256\M256Code\Exercises\Unit14`, contains a complete implementation of the class `HospitalClient`.

In summary, the problems demonstrated by the above exercise are as follows. To a client, the Hospital System should represent just one hospital. However, a client is able to create distinct coexisting coordinating objects, each with its own distinct core system, having its own `Team` objects, `Ward` objects, `Patient` objects and so on. Because the core systems are entirely separate, as illustrated in Figure 6, changes made via one coordinating object are not reflected in the other. This leads to inconsistencies which can result in information being lost when the system shuts down.

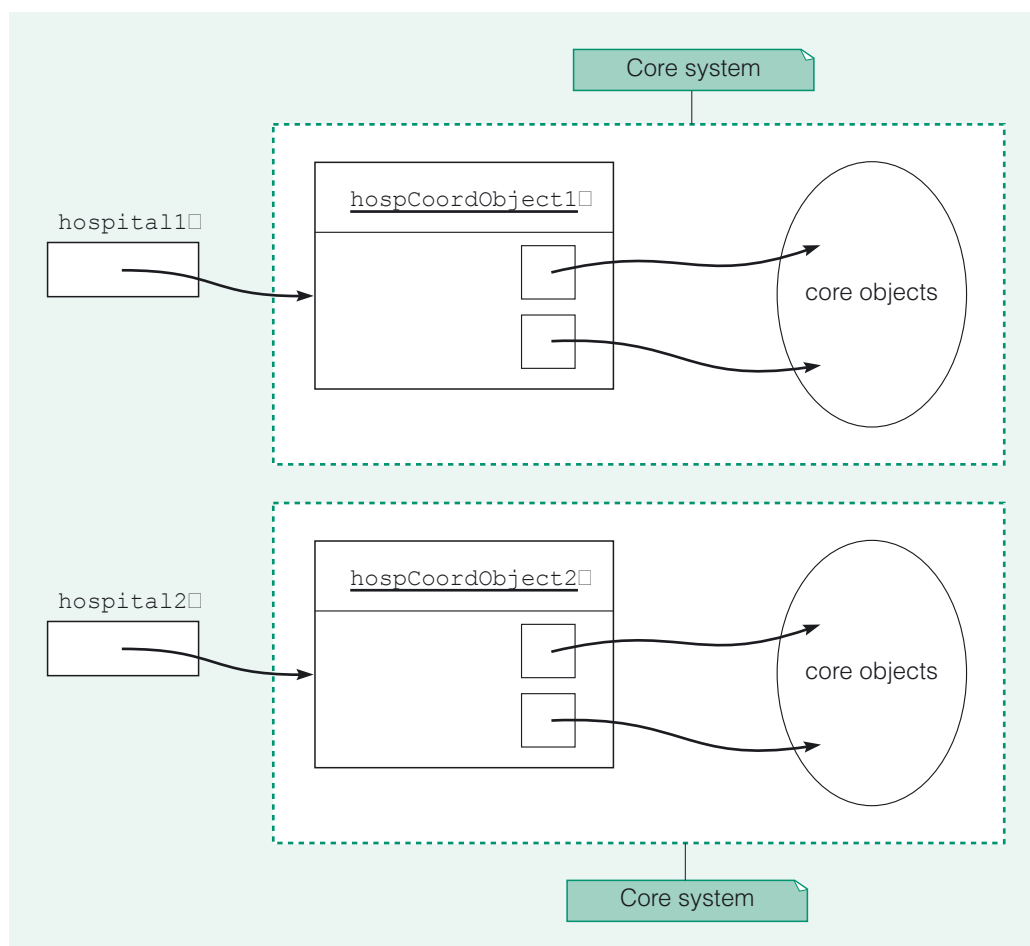


Figure 6 `hospital1` and `hospital2` reference different `HospCoord` objects, which in turn reference different core objects

Of course, it could be argued that a client should take care to create only a single coordinating object at a time. But a more robust approach would be for the core system to prevent a client inadvertently causing such problems, by ensuring that only one coordinating object can be created. This is where the **Singleton pattern** comes into play.

Singleton

Problem

Exactly one instance of a class (a **singleton**) is required.

Solution

Define a static method of the class that returns a new instance of the class if one does not exist; if an instance already exists, it simply returns that object. To make sure that clients cannot bypass the static method and create instances directly, do not make the constructor `public`.

Note that this pattern is not Java-specific: it generalises to other object-oriented languages which have the equivalent of Java's static methods.

Applying the Singleton pattern

We will show how application of the Singleton pattern can restrict a client to creating a singleton coordinating object of the class `HospCoord`.

Referring to the Singleton 'solution', the `HospCoord` constructor is already `private`, not `public`, and as you have seen there is already a static method, `getHospital()`, by which a client can obtain a `HospCoord` instance. Applying the Singleton pattern requires the code of that method to be amended. Here is the current version of that method, in full.

```
public static HospCoord getHospital()
{
    HospCoord hospital = null;
    try
    {
        FileInputStream fis = new FileInputStream("Hospital.data");
        ObjectInputStream ois = new ObjectInputStream(fis);
        hospital = (HospCoord)ois.readObject();
    }
    // if there is no file i.e. no previous details were saved
    catch(FileNotFoundException ex)
    {
        // invoke the constructor and thereby set up
        // hospital with the default initial state
        hospital = new HospCoord();
    }
    catch(Exception ex)
    {
        System.out.println("Initialisation error. You may need to
                           delete the data file.");
        System.exit(1);
    }
    return hospital;
}
```

As you saw previously, this `getHospital()` method always *creates* and returns a `HospCoord` object.

In the next exercise you will amend `getHospital()` so that if it is invoked a second time it does not create a `HospCoord` object but simply returns another reference to the object already in existence, as per the Singleton pattern, and as illustrated in Figure 7.

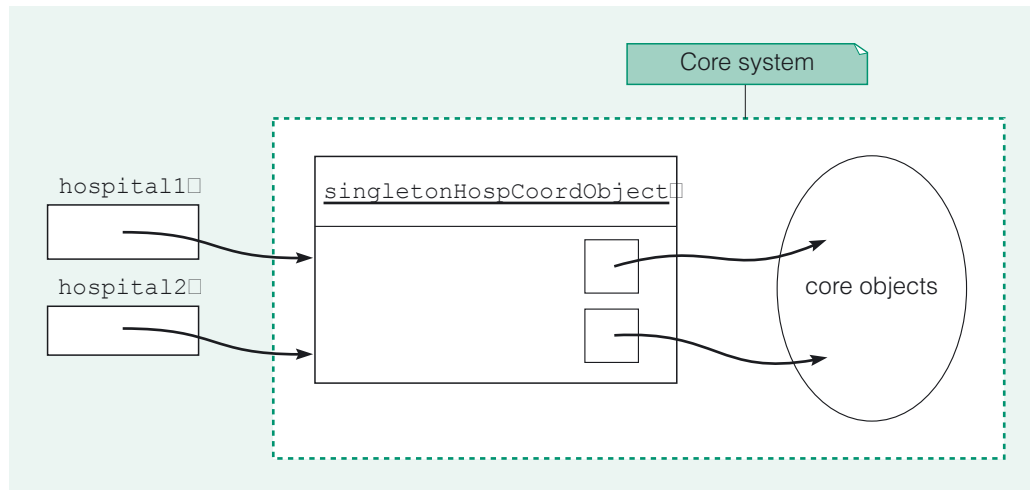


Figure 7 hospital1 and hospital2 both reference the singleton HospCoord object

Exercise 3



In NetBeans open the project `One_coordinating_object_Ex_3`, which is in the folder `M256\M256Code\Exercises\Unit14`.

Do not run the project yet, because this would cause the premature creation of the data file, `Hospital.data`. If you do accidentally run it, then before proceeding you should delete the data file `Hospital.data` from the project folder, as explained in Subsection 1.3.

The project consists of the three packages `hospitalcore`, `hospitalclient` and `m256people`, as in the solution to the previous exercise (`Two_coordinating_objects_Ex_2_Sol`).

Open the source code for the class `HospCoord` in `hospitalcore`. Note that its `getHospital()` method (positioned towards the end of the class definition) is as described above. Since the code for this class is rather lengthy we have folded up most of it, allowing you to concentrate better on this method.

- (a) To enable `getHospital()` to detect whether there is an existing `HospCoord` object, a reference to an existing object, if there is one, can be kept by using a static (class) variable, `hospital`. The idea is that, when a `HospCoord` object is created, `hospital` will reference it.

Add the following code to the variable declarations at the beginning of the `HospCoord` class definition.

```
private static HospCoord hospital = null;
```

Being a static variable, `hospital` will be accessible by the code of `getHospital()`.

- (b) Rewrite the code for the method `getHospital()` as follows.

The method should first check whether `hospital` references `null`.

- If so, then there is no `HospCoord` object currently in existence. The method should in this case attempt to set `hospital` to reference a new `HospCoord` object, whose core system details are obtained by reading in details from `Hospital.data` – just as before. If there are no saved details, `hospital` should be set to reference a new `HospCoord` object with default core system details, by invoking the constructor `HospCoord()` – again, just as before.
- If `hospital` does not reference `null`, then it already references an existing `HospCoord` object, so neither of the above actions is necessary.

At the end of the `getHospital()` method, `hospital` should be returned, as happens of course in the original version of the code. The difference is that now it will

reference either the new object, if one has had to be created, or the existing object, if one has been created previously.

(c) Run the project.

(d) Run the project again. What patient details were recorded from the previous execution?

Discussion.....

(a) The class definition should now begin as follows (changes are in bold).

```
public class HospCoord implements java.io.Serializable
{
    // links

    /**
     * a collection of all Ward objects
     */
    private Collection<Ward> wards;

    /**
     * a collection of all Team objects
     */
    private Collection<Team> teams;

    /**
     * the coordinating object
     */
    private static HospCoord hospital = null;

    ...
}
```

(b) Here is the rewritten code. There is actually very little difference from the original (changes are in bold). Instead of `hospital` being a local variable, initially set to `null` within the method, it is now a static variable which is tested to see whether it references `null`.

```
public static HospCoord getHospital()
{
    if (hospital == null) //if a coordinating object does not already exist
    {
        try
        {
            FileInputStream fis = new FileInputStream("Hospital.data");
            ObjectInputStream ois = new ObjectInputStream(fis);
            hospital = (HospCoord)ois.readObject();
        }
        // if there is no file i.e. no previous details were saved
        catch(FileNotFoundException ex)
        {
            // invoke the constructor and thereby set up
            // hospital with the default initial state
            hospital = new HospCoord();
        }
        catch(Exception ex)
        {
            System.out.println("Initialisation error. You may need to
                                delete the data file.");
            System.exit(1);
        }
    }
    return hospital;
}
```

(c) The output should be as follows.

```
hospital1 and hospital2 are the same objects.
```

```
Initial details of hospital1:
```

```
[details omitted]
```

```
Initial details of hospital2:
```

```
[details omitted]
```

```
Initial patient details of hospital1:
```

```
[]
```

```
Initial patient details of hospital2:
```

```
[]
```

```
Admitting a patient to hospital1...
```

```
Final patient details of hospital1:
```

```
[Ms Bet Lynch female 23/05/78]
```

```
Final patient details of hospital2:
```

```
[Ms Bet Lynch female 23/05/78]
```

This demonstrates that `hospital1` and `hospital2` reference the same object, and that, as you would then expect, the effect of admitting a patient to the hospital represented by `hospital1` is mirrored by `hospital2`.

(d) Running the project again reveals that the details of the patient admitted during the previous execution of the system were saved. (The code then goes on to readmit that patient – but this is not the point here.) The key part of the output is the initial part:

```
Initial patient details of hospital1:
```

```
[Ms Bet Lynch female 23/05/78]
```

```
Initial patient details of hospital2:
```

```
[Ms Bet Lynch female 23/05/78]
```

The project `One_coordinating_object_Ex_3_Sol`, which is in the folder `M256\M256Code\Exercises\Unit14`, contains a complete solution to this exercise.

Exercise 3 demonstrates that the problems noted in Exercise 2 are solved by implementing the coordinating object as a singleton.

However, this solution only prevents a single user at a single machine creating multiple coordinating objects. There would still be problems for a system which is deployed on multiple machines, each of which would have a distinct instance of the `HospCoord` class. In Section 4 you will meet a solution to this problem.

3.4 The Publish–Subscribe pattern

The **Publish–Subscribe pattern** is deeply embedded in the Java API, particularly in its event-handling capabilities.

Publish–Subscribe

Problem

Different objects (subscribers) all need to react to certain changes in another object (the publisher), to which they are otherwise unrelated.

If you are familiar with Java multithreading, you may have spotted another potential problem. Suppose two threads execute `getHospital()` at about the same time. The first thread might check and find no coordinating object in existence yet. At that point the second thread might take over and also find no coordinating object exists. Both threads would then go on to create separate coordinating objects.

There are various ways to implement the Singleton pattern so that this problem cannot occur, but a discussion of these is beyond the scope of this course.

Solution

The publisher maintains a list of subscribers, which a new subscriber can 'register' with. When the publisher changes, it sends a notification – the same message – to each object on the list. Each subscriber receives the same message, but of course may react in its own way. There is no need for the publisher to adapt to individual subscribers: it simply broadcasts the notification, and any registered subscribers receive it.

In this solution, there is coupling between the publisher and the subscribers, but it is a loose coupling via a list, with the publisher remaining independent of the details of the subscribers. Indeed, there may at times be no subscribers on the list, and the publisher would not care about this.

In Java, each GUI component (button, text field etc.) maintains a collection of listener (subscriber) objects. All that a listener is required to do is implement a particular interface, which means that it responds to certain messages. It will then be notified using one of these messages when the component changes (that is, when an event occurs such as the button being pressed, text being entered etc.), and it can take its own appropriate action.

Exercise 4

In fact, this code is a slightly simplified version of that in `HospGUI`, which was produced using NetBeans GUI designer facilities.

Below are some code extracts from the user interface class (`HospGUI`) in the Hospital System. The publisher object in this case is the `JButton` referenced by `treatButton`. The 'change', i.e. the event of interest, is the pressing of the corresponding button in the user interface.

By examining the extracts and considering what will happen when the system is running, answer the following questions.

- Which object is a subscriber in this example?
- By what means is the subscriber registered?
- What notification message is sent by the publisher object to its subscribers when it changes (i.e. when the button is pressed)?
- What happens when the subscriber in part (a) receives such a notification message?

First code extract:

```
treatButton = new JButton();
```

Second code extract:

```
private class TreatButtonWatcher implements ActionListener
{
    public void actionPerformed(ActionEvent evt)
    {
        recordTreatment();
    }
}
```

Third code extract:

```
TreatButtonWatcher aWatcher = new TreatButtonWatcher();
treatButton.addActionListener(aWatcher);
```

Fourth code extract:

```
/**
 * This method calls on the coordinating object to record
 * the treatment of a patient.
 */
private void recordTreatment()
{
    [code omitted]
}
```

Discussion.....

- (a) The subscriber is the `TreatButtonWatcher` object referenced by `aWatcher`.
- (b) The subscriber is registered by being provided as an argument to the message `addActionListener()`, which is sent to the publisher.
- (c) When the publisher (the `JButton` referenced by `treatButton`) changes due to the button being pressed, it sends to each of its subscribers the message `actionPerformed()`, with a suitable `ActionEvent` object as argument. The method `actionPerformed(evt)` is the only method implemented in the class `TreatButtonWatcher`.
- (d) When `aWatcher` receives a notification message, a message `recordTreatment()` is sent to the user interface object, which calls on the coordinating object to record the treatment of a patient.

The above exercise illustrates how the Publish–Subscribe pattern is used to inform the core system of events in the GUI. In Section 4 you will see how this pattern can be applied to keep a user interface up to date with changes to the core system.

In this section you have been introduced to the concept of a design pattern, and have looked at three design patterns of particular significance for the Hospital System.

4

Multi-user systems

In this section we explore some of the implications of the fact that a real-world hospital administration system would have to be a **multi-user system**. Even a small hospital is a very complex affair, which could not possibly be managed if the administration system were confined to a single computer that only one person at a time could use. So it must be possible for multiple administrators to work simultaneously on different computers, that is, to have concurrent access to the system. However, all the computers must be in communication with one another, as otherwise the administration would break down. For example, if an administrator admits a new patient at one computer, the patient's information must then be accessible to the computers being used by all the other administrators.

This leads to the idea that the computers involved must all be on a computer network and thus form what is called a **distributed system**.

So the two main aspects of a multi-user system that we will consider here are as follows.

- 1 The system is distributed.
- 2 Access to the system is concurrent.

The concepts involved in distribution and concurrency are very important, and you will undoubtedly meet them again if you pursue studies or work in computing.

The system must also be persistent, so that the information it holds will continue to exist even when the software is not being executed. We will briefly discuss one form of persistence provided by Java.

In this section you will be carrying out some short practical exercises which illustrate the above aspects. These are purely to demonstrate distributed concurrent systems in action; you are not required to understand, or even look at, the code, which uses Java libraries that are beyond the scope of this course.

4.1 Distributed systems

A crucial aspect of the design of a distributed system is its **deployment**: how the various constituent components are physically arranged, or in other words, what physical computer or computers each part of the software runs on and how these computers will communicate.

This is not the same thing as the software's logical architecture, which is to do with how the major software components are related to one another *logically*. Deployment is concerned with where running software is located *physically*. Of course the two things cannot be divorced from one another, and the deployment has to take the logical architecture into account. For example, if one component is dependent on another but the two are placed on different physical computers, then the computers concerned must be connected to one another.

You saw earlier that the Hospital System has a two-tier architecture, generally referred to as a client-server architecture. The user interface (the GUI) is a client, which requests services of the core system, which is a server. The client is dependent on the server.

Exercise 5

- (a) Bearing in mind the fact that the system must be a multi-user one, with the Hospital System user interface running on many computers at the same time, but with all the users ‘seeing’ the same hospital, what do you think would be a suitable way to deploy the components of the Hospital System? (We are not expecting anything complicated here – just a short, common-sense answer.)
- (b) Where would the coordinating object be placed?

Discussion.....

- (a) There could be multiple copies of the client software (the Hospital System GUI) each running on a separate client computer, and just one copy of the server software (the Hospital core system) running on a server computer. All the client computers would be connected to the server computer.
- (b) Since all the clients should work with the same coordinating object, this object should be located on the server computer.

The deployment arrangement discussed in Exercise 5 can be illustrated in a **deployment diagram**.

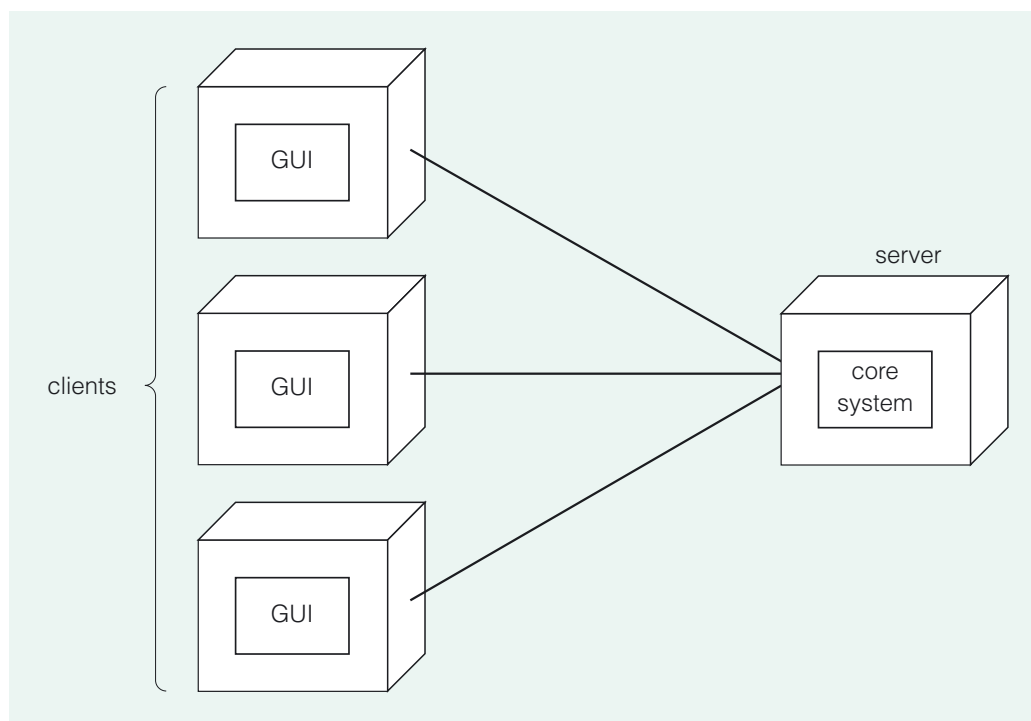


Figure 8 A deployment diagram for a distributed Hospital System

Like classes, components can be thought of as having instances. A copy of a software component such as the Hospital System GUI can be deployed to, and run at, many different locations, with each running copy being an instance. Only three GUI instances are depicted in the diagram, but of course there may be many more in reality. There is only one coordinating object, which is a singleton object located on the server.

You can contrast this distributed system with the situation discussed at the end of Subsection 3.3. There you learnt that even when the Singleton pattern is applied to the Hospital System, a problem still arises if the core system is deployed on multiple computers: in that situation multiple coordinating objects are created. In contrast, the deployment we are considering here overcomes the problem by locating the core

system at the server end: there is only one server, and it creates a shared coordinating object that is unique within the system.

Persistence

As mentioned previously, the system must be persistent. Our implementation of the Hospital System achieves persistence by saving the state of all the objects in the system when it is shut down. When the system starts up again, this information about all the objects' states is used to regenerate the system with new objects which are in exactly the same state as the old objects.

For a Java object's state to be saved and recreated as described above, the object must be **serialisable**. That is, the object's class must implement the `Serializable` interface, which all the relevant classes in the Hospital System do. Here is the declaration of `Team` for example:

```
public class Team implements java.io.Serializable
```

Most Java interfaces specify methods that must be implemented by any class that implements the interface concerned. However, `Serializable` specifies no methods: its role is simply to inform the JVM that objects of the class in question can be represented in a way that enables them to be stored in a file (i.e. persist) or sent over a network. The programmer is not obliged to do anything beyond declaring that the class implements `Serializable`. The programmer must then implement some mechanism for transferring information about objects' states to and from some storage device, that is, for filing the information in and out.

Object serialisation is just one way of facilitating persistence, and there are many other kinds of storage, such as databases, which offer much more sophisticated ways of storing data. However these fall outside the scope of M256.

Communication between components

We also need to consider how the components in Figure 8 will communicate with one another. Just as there are many different forms of persistence, there are also many different formats for communication. The one used by the example system you will look at in the exercises later in this section is called **Remote Method Invocation (RMI)**. This is a Java mechanism that allows a program to send messages to a Java object which is in a different JVM on a different computer. Any method arguments and return values are exchanged over the network. If the values concerned are objects, then for them to be sent over a network, RMI requires that they are serialisable; this is the same property that is needed to facilitate the simple persistence mechanism of storing in a file, which we looked at above.

4.2 The Remote Clinic System

We will now introduce you to a small system of the type we have been describing. It will have multiple GUI clients, communicating concurrently with a core system running on a server. Communication between the components will be via RMI, and the core system will use serialisation to facilitate both object persistence and the sending of objects over the network between components. This miniature system could be run on multiple computers, like the system shown in Figure 8. We cannot ask you to do this since you may have only one computer, so in the exercises everything will be run on one machine, but remember that if you did have access to many networked computers, each component could be deployed to a separate machine.

In the exercises we will first demonstrate that all the clients in the system are interacting with the same coordinating object. Then we will go on to explore two issues that commonly arise when two or more clients access a system concurrently.

The first issue is that if two or more clients try to make changes at the same time, problems may occur. To give just one example, imagine a distributed system for selling theatre tickets. Suppose two clients try to reserve a seat for a particular evening, but only a single seat is in fact available. If the clients both access the system at more or less the same time, they may both find there is a free seat available and go ahead and make a reservation, leading to the same seat being sold twice.

The second issue is that even clients that are not trying to change the state of the core system will normally want to be kept informed of its state. In particular, there needs to be a way for all the clients to be informed whenever one client makes a change.

We begin by describing the system. We ask you to imagine a clinic which provides some health-related service – for example, advice on nutrition. Clients who wish to attend the clinic have a booking made for them by an administrator. The clinic serves a very wide community and so there are many administrators based at different locations. The clinic is going to be given a computerised booking system, and each administrator will have a computer from which they can connect to the clinic and make bookings.

As a reminder that it is a distributed system we have named it the Remote Clinic System.

The system you will see is an early prototype with only two very simple use cases.

- ▶ *Add Booking* which adds a person's name to a list of bookings.
- ▶ *Get Bookings* which returns a list of all the current bookings.

The use cases have corresponding coordinating methods, documented as follows.

addBooking

```
public void addBooking(Name name) throws java.rmi.RemoteException
```

Makes a booking for the person.

Parameters:

name – the name of the person

Throws:

java.rmi.RemoteException

getBookings

```
public java.util.List<Name> getBookings() throws java.rmi.RemoteException
```

Returns a list of bookings.

Returns:

a list of Name

Throws:

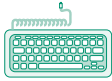
java.rmi.RemoteException

For simplicity this prototype is not concerned with dates and times. You can imagine that these details will be added later in a subsequent iteration of the development process.

Name, in this system, is a very simple class representing people's names as first name and second name.

Both of these methods are specified as throwing a `RemoteException` because Java requires this of any method that is going to be invoked on a remote object.

This describes the entire behaviour of the prototype Remote Clinic System. Although it is a fairly minimal system, it does all we require for demonstration purposes. In the next exercise you will run the system and experiment with it. (Note that the exercise is self-explanatory and there is no separate discussion).



If you run the client when the server is not running, the client will simply exit.

Exercise 6

In NetBeans open the projects `RMI_Server_Ex_6` and `RMI_Client_Ex_6`, both of which are in the folder `M256\M256Code\Exercises\Unit14`.

- (a) Since the client GUI objects depend on the coordinating object, and the coordinating object is on the server, you must begin by running the server project `RMI_Server_Ex_6`. Right-click on this project in the Projects window and choose **Run Project**.

In the NetBeans Output window the message

```
Saved data not found – creating fresh object.
Clinic Server running...
```

should appear. This means the server has successfully started, and a remote coordinating object is ready to receive messages from clients.

- (b) Next run the client project `RMI_Client_Ex_6`. A simple GUI should appear, similar to the one shown in Figure 9.

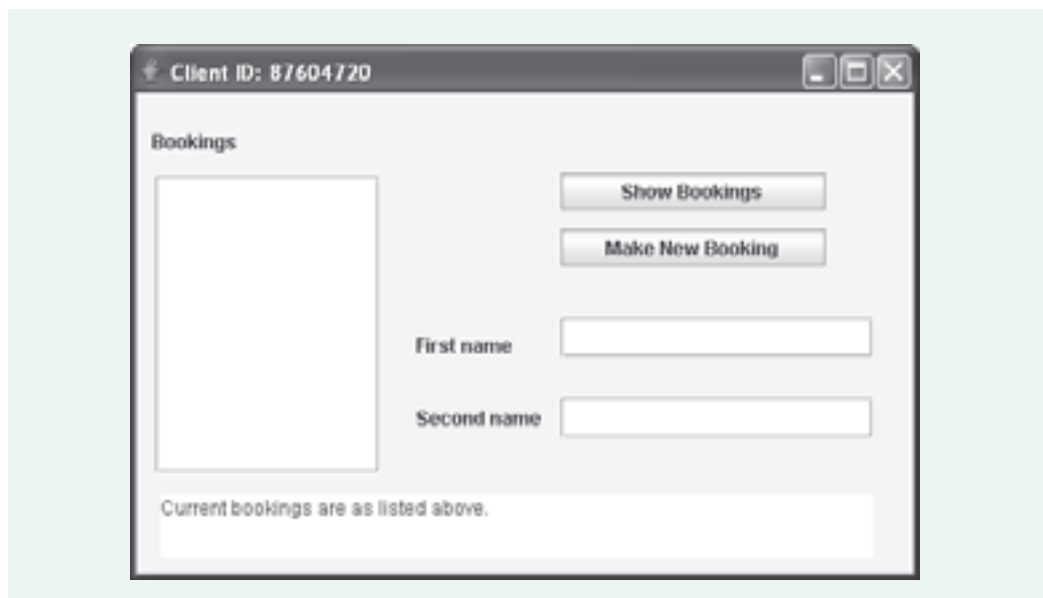


Figure 9 A GUI for the Remote Clinic System client

So that clients can be distinguished, each is assigned an ID number chosen at random; this is displayed as the title to the GUI window. An ID number can be up to eight digits long, so any two clients are highly likely to have distinct IDs, although this is not entirely guaranteed. The number is shown as 87604720 in Figure 9, but the one on your GUI will almost certainly be different.

- (c) Enter a first name and a second name in the relevant boxes, and then click on **Make New Booking**. This will send the appropriate message to the coordinating object. A dialogue box will appear, asking you to confirm your wish to make a booking. Click on **Yes** and the new booking will appear in the **Bookings** list, as in Figure 10.

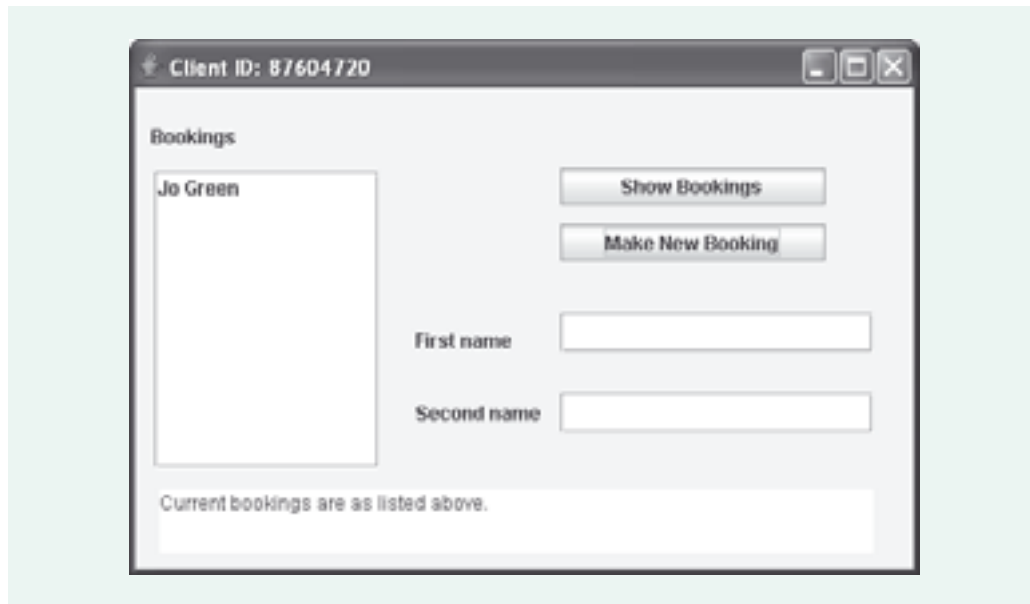


Figure 10 A new booking

- (d) Now leave this GUI running and run the project `RMI_Client_Ex_6` again to create a second client.

When you click on NetBeans again to run the new client, the order of the windows will change and the first client will be hidden. Arrange the client windows so that you can see both of them side by side.

When both clients are visible, you will find that the new client already shows the booking you just made. This is because when it is first displayed it asks the coordinating object for a list of bookings. There is only one coordinating object, which both clients share, so the second client finds the existing booking.

- (e) You can now experiment with adding bookings from either client, running additional clients and so on. Every time a booking is added by one client, clicking the Show Bookings button in the other clients will reveal that they are all interacting with the same coordinating object. (For the moment the information is only updated when you click on the Show Bookings button, but later in this section you will see how to arrange for all the clients to receive automatic updates whenever any one of them adds a booking.)
- (f) When you have finished experimenting you should shut down all the clients by closing their windows. Once the clients are all shut down, stop the server as follows:
- (i) Click on the `RMI_Server_Ex_6` (run) tab in the NetBeans Output window (but make sure you do *not* click the small close button  which would close the tab).
 - (ii) Type `exit` in the Input field that appears just under the Output window, and press Enter on your keyboard.

You should see a message that the server is stopping. When the server stops it saves the state of the Remote Clinic System.

If the server is not stopped, it will cause a conflict when you try to run the server in the next exercise. Only one RMI server can run at a time.

If you have problems stopping the server, refer to 'Halting a running program' in Subsection 3.1 of the *NetBeans Guide*.

4.3 Concurrent systems

You have seen that it is possible for multiple clients, which in the case of Exercise 6 were all running on the same machine but which in reality could be running on separate machines, to access a unique shared coordinating object which is on a server. Having a single coordinating object solves the problem of how all the clients can see the same core system which they have in common. However, whenever clients share data another kind of problem arises.

Suppose the rules of the Remote Clinic System were altered to specify that the maximum number of bookings allowed was ten.

You have met similar requirements in other systems. What a client should do is check the number of bookings, and only go ahead if it is acceptable to do so. However, the coordinating object should also ensure that the invariant on the system (corresponding to the constraint that the number of bookings is no more than ten) is preserved, in case for example a faulty client does not make the check. So when asked to make a booking, the coordinating object should itself first check that the current number of bookings is less than ten. Only if this is the case should the coordinating object, `clinicCoord`, proceed to make the new booking. The sequence diagram in Figure 11 shows such an interaction.

Figure 11 shows the user interface (the client) obtaining a collection of bookings and determining whether there are fewer than ten of them. An alternative design would be to have the user interface send the coordinating object, `clinicCoord`, a message asking for just the number of bookings. However, since the list of bookings displayed in the user interface should be updated in any case, it is convenient to get the collection of bookings from `clinicCoord`.

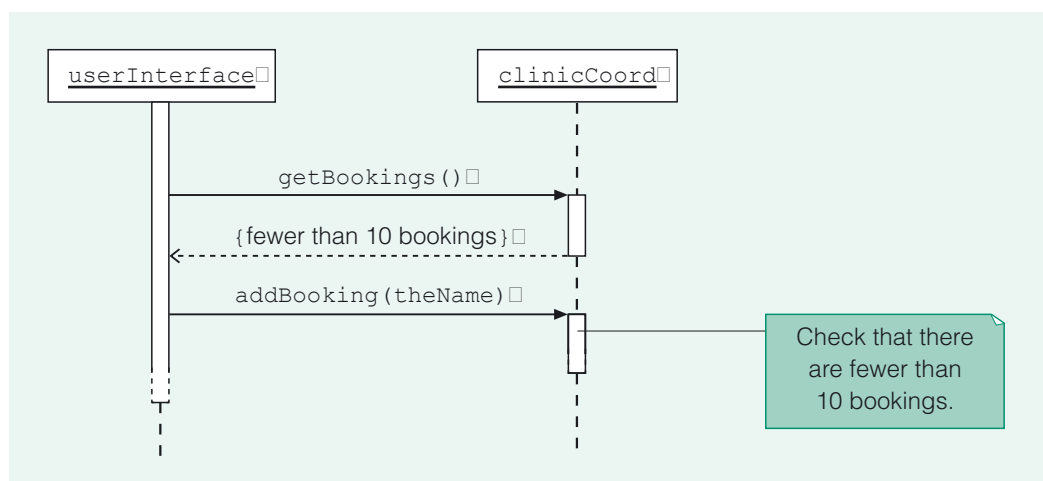


Figure 11 Making a booking

This 'check and act' approach is fine if there is only one client. But in a system like ours, where there are multiple concurrent clients, two clients – `client1` and `client2` say – might try to make a booking at the same time. In this situation, the execution of the coordinating method `addBooking(aName)` which is initiated by `client1` could get interrupted by the execution of the same method `addBooking(aName)` initiated by `client2`.

This can happen because on the server (assuming this is a single-processor computer) processor time is shared between separate executions like these on a rotating basis, allowing each a share of the processor's time. Usually, this happens so quickly and seamlessly that it appears to the user as if the separate executions are being carried out simultaneously.

But problems can arise – for example, when the switching happens in the middle of the execution of a method. Consider the code, below, of the coordinating method `addBooking(aName)`, which carries out the actions of checking and booking that were described above Figure 11. This is the method that the coordinating object executes when requested to make a booking.

```

public void addBooking(Name aName) throws RemoteException
{
    if (bookings.size() < 10)
    {
        bookings.add(aName);
    }
}

```

Suppose that there are nine current bookings and that two clients, `client1` and `client2`, each having checked the number of current bookings, both make a request at about the same time for the coordinating object to make a booking. Two executions of the `addBooking(aName)` method are initiated, one per client.

Suppose that `client1`'s execution gets the processor first, and that in the consequent execution the coordinating object checks the number of current bookings and, having discovered that there are only nine of them, is about to make the new booking, when that execution is interrupted by `client2`'s execution.

Now `client2`'s execution has the processor, and in the consequent execution the coordinating object checks the number of bookings, discovers that there are only nine of them, and so proceeds to make the new booking: `client2` has made its booking.

The processor passes back to `client1`'s execution, and processing resumes where it left off. The coordinating object, on the basis of what is now out-of-date information, continues to make the new booking: `client1` has also made its booking – and there are now eleven bookings!

In the next exercise you will see a simulation of this problem (although in this exercise, for reasons you do not need to understand, the first client rather than the second client completes its booking first). As in Exercise 6, you will run two clients with the same server. You will make a request with each of these clients, at about the same time, for the coordinating object to make a booking, and you will see that something like the situation described above occurs – despite the coordinating object attempting to check the number of bookings, the maximum is exceeded, and eleven bookings are made.

This is a simulation because in the implementation we have artificially introduced a delay into the coordinating object's processing of the method `addBooking(aName)`. With multiple users it is quite possible that two users might hit the button to make a booking at more or less the same instant, leading to the problem described above. However, with just one user – you – the processing of the method would otherwise be too quick. Having hit the button in one client to make a booking, the processing of this request would probably be completed before you could hit the button of the other client to make a booking. In order to see the problem, everything needs to be in slow motion, which is what the artificial delay achieves.

As with Exercise 6, the next exercise is self-explanatory and does not have a separate discussion.

Exercise 7

Open the projects `RMI_Server_Ex_7` and `RMI_Client_Ex_7`, both of which are in the folder `M256\M256Code\Exercises\Unit14`.

- Start the server project, as you did in Exercise 6. This time you should see a message saying saved data has been found.
- Now run two clients. As before you will need to arrange the client windows so that you can see both of them side by side.



Remember that only one RMI server can run at a time – if the server from Exercise 6 was not stopped, it will cause a conflict here. In the case of a conflict, return to the instructions at the end of Exercise 6 and stop the previous server.

You will find that nine bookings have already been made, so there is only one free space left. You will attempt to double-book this one remaining space, by requesting a booking in one client, and then requesting a booking in the other before the processing of the first client's request is complete. You will need to follow the instructions below carefully.

- (c) In your first client enter a first and a second name, and click on Make New Booking. This will bring up a dialogue box as in Figure 12. Do not click on Yes yet.

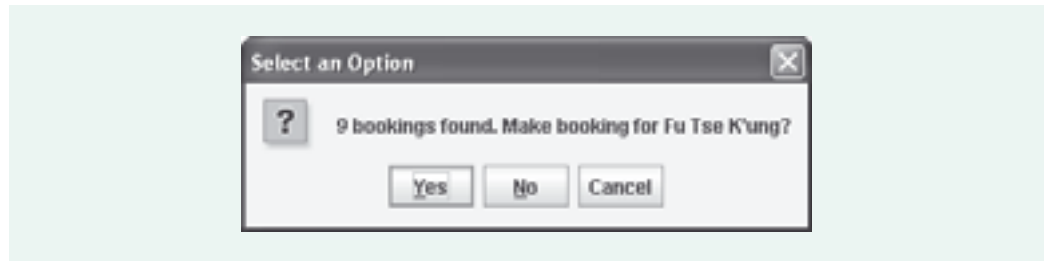


Figure 12 Confirmation dialogue

The first client has not yet submitted its booking request to the coordinating object because it awaits confirmation from the user.

- (d) Now switch to the other client, enter a first and second name, and click on Make New Booking. This will again bring up a dialogue box. Do not click on Yes yet.
- (e) Now return to the first client and click on Yes in the dialogue box. Then quickly (you have about 10 seconds to do this!) go to the second client and click on Yes in the dialogue box.

There now follows the delay that was mentioned above, during which a 'ghost' appears in place of each dialogue box.

After a few seconds you should see that the bookings list in the second client is updated to show that eleven bookings have been made. Clicking on Show Bookings in the first client (which at this point shows ten bookings) then updates the first client's list to show the eleven bookings.

If you look at the output for the server (you will need to click on the RMT_Server_Ex_7 (run) tab in the Output window of NetBeans) you will see that both executions of `addBooking(aName)` 'believed' there were fewer than ten bookings and therefore went ahead and made a booking.

If you want to experiment further with this project once the list is full, you can stop the server, delete the `.data` file from the `RMI_Server_Ex_7` project folder, and then restart the server, which will create a new, empty collection of bookings. Alternatively, if you terminate the server instead of exiting normally (following the instructions under 'Halting a running program' in Subsection 3.1 of the *NetBeans Guide*), the changed state of the system will not be saved, so you can easily run this exercise again.

Problems like that exemplified in Exercise 7 abound wherever concurrent executions operate on shared data, and the study of how they can be prevented is a major topic in computing. The basic idea is that there must be some form of **concurrency control**: access to the shared data must be regulated to prevent the situation described above from occurring.

There are many forms of concurrency control. We will describe one simple scheme, without exploring how it is coded, and demonstrate that it successfully prevents overbooking.

Locking

The idea is that when the `addBooking(aName)` method of the coordinating object is invoked by a client, the coordinating object **locks** itself. While the object is locked no other client can invoke the `addBooking(aName)` method until the lock is released.

To achieve this, the method `addBooking(aName)`, which the client invokes to make a booking, is altered so that when the coordinating object receives the corresponding message, it carries out the following sequence of actions.

- 1 Locks itself.
- 2 Checks the number of bookings.
- 3 If the number of bookings is less than 10, makes the new booking.
- 4 Unlocks itself.

If the coordinating object is already locked because it is making a booking for one client, and another client tries to invoke `addBooking(aName)`, the second client is made to wait until the first booking attempt has finished and the coordinating object is unlocked.

This means that the situation simulated in Exercise 7 cannot happen. With the first client already having made a booking request, when the second client also requests a booking the first client's request is first entirely completed. So the first client's booking is made before the second client's request is dealt with. When the second client's request is processed, it is discovered that there are ten bookings and this request is denied.

In Exercise 8 you will work with a new version of the Remote Clinic System which puts into practice the scheme described above.

Exercise 8



Open the projects `RMI_Server_Ex_8` and `RMI_Client_Ex_8`, both of which are in the folder `M256\M256Code\Exercises\Unit14`. The coordinating method `addBooking(aName)` now implements the locking of the coordinating object as described above.

Start the server and then run two clients. There should initially be nine bookings. Attempt to overbook by carrying out exactly the same sequence of actions as in Exercise 7.

Discussion.....

You should find that the attempt to overbook fails. Only the booking of the first client succeeds.

If you look at the output for the server (click on the **RMT_Server_Ex_8 (run)** tab in the **Output** window of NetBeans), you will see that the first client's booking succeeded because the first execution of `addBooking(aName)` found that the bookings list was not full, but the second client's booking failed because the second execution of `addBooking(aName)` found that the bookings list was full.

Keeping up to date

As previously mentioned, the clients in the Remote Clinic System can see changes made by other clients, but not until the user clicks on the **Show Bookings** button to refresh the list of bookings. It would be preferable for changes to be shown as soon as they happen.

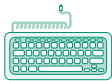
One possibility would be for the GUI to keep reloading the list of bookings at regular, frequent intervals. If this happened often enough – several times a second say – it would seem to the user that changes were known about immediately.

However, this would be highly inefficient, because changes are relatively infrequent. Most of the time the client would simply be reloading a list which had not actually changed, and effort, processor time and network resources would be wasted.

A better solution is to use the Publish–Subscribe pattern, which was described in Subsection 3.4. Each GUI client ‘signs up’ with the coordinating object and, when the latter changes, it sends all the clients a message telling them that there has been a change. They can then all refresh their lists of bookings. This is much more efficient because the lists are only refreshed when there has been a change. The final version of the Remote Clinic System puts this idea into practice.

In more detail, what happens is as follows.

- ▶ When a client is created it registers itself with the coordinating object, i.e. it subscribes.
- ▶ Whenever a new booking is made, the coordinating object notifies all the clients, i.e. it publishes the fact that there has been a change.
- ▶ When the clients receive the notification, each one sends the coordinating object a message asking for the current bookings, and then uses the message answer to update its list of bookings.
- ▶ When a client is closed, it unsubscribes itself before exiting. This is important, as otherwise the coordinating object would send notifications to clients which no longer existed. The ensuing exceptions would be caught and dealt with without the server crashing, but it is far preferable for the clients to unsubscribe themselves in an orderly manner.



Exercise 9

In NetBeans open the projects `RMI_Server_Ex_9` and `RMI_Client_Ex_9`, both of which are in the folder `M256\M256Code\Exercises\Unit14`. These projects employ the Publish–Subscribe pattern as described above.

Start the server and then experiment with running two or more clients.

There should initially be no bookings. Notice that the `Show Bookings` button has been removed, since it is no longer needed.

Discussion.....

You should find that when you make a booking through one client, all clients are instantly updated to list the new booking.

In this section you have been introduced to the concepts of concurrency, distribution and (briefly) persistence, and have seen practical illustrations of some of the issues involved.

5

New ways of developing software

In this concluding part of the course we aim to expand your understanding of software development, and place in context the knowledge and skills you have gained in M256, by discussing some relatively new ways of developing software.

This section draws on Chapter 8 of M882 *Managing the software enterprise*.

M256 has introduced you to what are generally recognised to be key phases of object-oriented software development: phases such as conceptual modelling, dynamic modelling, implementation, testing etc. You have also been introduced to some key concepts and techniques associated with each of these phases: the textual analysis of a requirements document, the use of sequence diagrams etc. In order to simplify your introduction to software development, we have largely discussed each of these topics separately, taking care to stress that in reality the phases might be combined in a variety of ways, with their associated activities potentially being employed in different ways and in different combinations.

Having studied M256, you should now have a broad base of knowledge, skills and techniques that can be applied in a wide variety of software development projects. Professional software developers working for a development company or department will typically have a similar broad base of knowledge, which they apply – along with their experience – to whatever particular method of development is required of them.

Back in *Unit 1* we outlined two broad categories of software development method, that is, two distinct ways of developing software: waterfall and iterative. In a waterfall method, there is a largely linear progression to the development, with each development phase being completed before the next starts; in an iterative development, phases are repeated cyclically. Iterative methods have become a dominant force in the systematic development of software, research having shown that they tend to result in better quality software.

SAQ 12

List some specific advantages of iterative methods over waterfall methods.

ANSWER.....

Here are some advantages that were discussed in *Unit 1*, Subsection 5.3. Iterative methods:

- ▶ typically produce a series of prototypes, giving the client opportunities to 'see' what is being developed throughout the process;
- ▶ typically include many iterative rounds of testing, enabling problems to be resolved early on, and generating confidence in the completed system;
- ▶ allow, through regular reviews, changes or additions to the requirements to be better managed as the project progresses;
- ▶ allow developers to return to previous phases to revise earlier decisions.

There is no single iterative method, and in practice there is great variety in the methods that are described as iterative. In this section we will outline three kinds of iterative approach – the Rational Unified Process, rapid application development and agile development – before considering a very different way of developing software: open source development.

5.1 The Rational Unified Process

Object-oriented development methods rose to prominence during the 1980s, with many leading software development experts advocating their own methods. The Rational Software Corporation was founded in 1981 and recruited three of the prominent experts of that time: James Rumbaugh, Ivar Jacobson and Grady Booch. These 'three amigos', as they were known, were involved in developing first the UML for the representation of object-oriented software models, and then the **Rational Unified Process (RUP)** to prescribe how software should be developed using the UML.

The RUP has evolved over the years and is not a single method for developing software but a description of a number of methods which have in common a set of 'best practice' elements (Rational, 1998), described in outline as follows. These elements have become key parts of many object-oriented methods of software development, and the ideas should be familiar to you from M256.

► Iterative development

The RUP supports an iterative approach that addresses the highest-risk items at each stage. Each cycle ends with a working release of the system, which is evaluated by users. Each cycle is divided into four stages:

- 1 *Inception*, in which the business case for the system is established and the project scope is defined.
- 2 *Elaboration*, in which the system domain is analysed, an architectural foundation for the system is established and the project plan is developed.
- 3 *Construction*, in which components are developed and integrated into the product and all features are thoroughly tested.
- 4 *Transition*, in which the current system release is transferred into its working environment and users are trained to use it.

► Management of requirements

The RUP describes how to elicit, organise and document requirements using the notions of use case and scenario.

► Use of components

An early focus of the RUP is on developing the outline logical architecture of the system, thus promoting the use of new and existing components.

► Visual modelling

The RUP involves visual (diagrammatic) modelling of software at all stages to help communication among developers and clients, to assist in viewing the system at appropriate levels of abstraction and to maintain consistency between design and code.

► Verification of software quality

The RUP builds assessment of the evolving system into all activities of the process, in respect of functionality, reliability and performance. Such quality assessment involves all participants and uses objective measurements and criteria.

► Management of change

The RUP describes how to control, track and monitor changes to all artefacts of the development – code, models, documents etc. – to facilitate successful iterative development.

5.2 Rapid application development

Developing complicated systems is a highly complex business. Development phases and techniques of the kinds you have learnt about go a long way to easing the development of complex systems, rendering more manageable both the process and the software itself, but in reality all kinds of factors may impact on the development process and potentially throw it off course. The most significant areas that can be affected are:

- ▶ keeping within budget;
- ▶ keeping within the development schedule;
- ▶ creating software that meets all the requirements.

Any software development project must have as its key aim to deliver a stated set of requirements within some cost and within some timescale. However, in some circumstances (e.g. poorly defined projects, chaotic software processes, lack of experience with similar projects) it may not be possible to accurately predict either the cost or the timescale for delivering a system with a given set of requirements. In these circumstances, what typically happens is that the delivery date is moved on, and costs increase, in order to deliver the agreed requirements. Some people find it helpful to imagine a triangle with these three elements – requirements, costs and timescale – as vertices, as shown in Figure 13. This simply serves as a reminder that these three factors are interrelated.

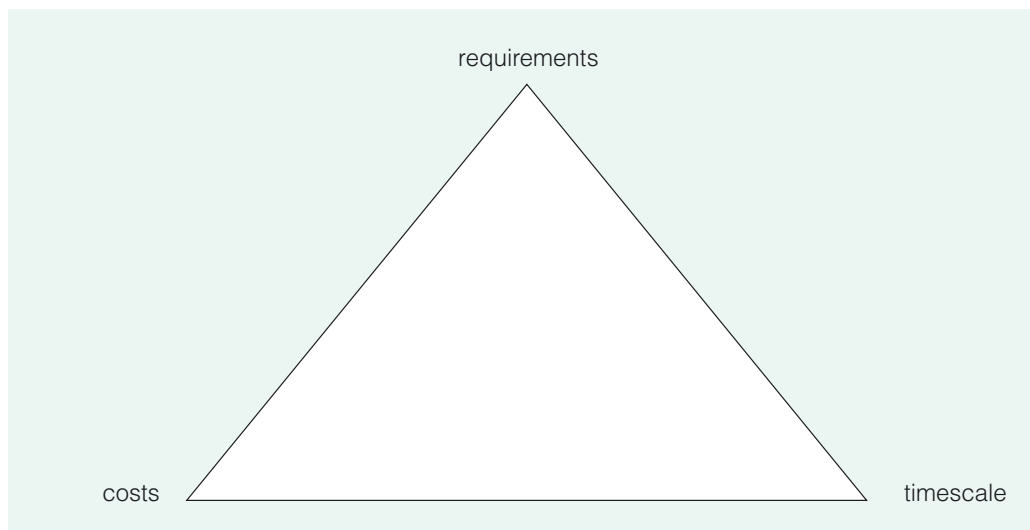


Figure 13 Requirements–costs–timescale triangle

However, iterative development does provide a way to retain control over requirements, costs and timescale, by making it easier to vary the requirements instead of the costs and timescale – in other words, to deliver what is possible by the date agreed and within the costs agreed.

This is essentially done by deferring requirements from the current iteration to some later iteration. For the current iteration, the focus is on the currently most important requirements, with those that are less important being postponed. This leads to the idea of **timeboxing**, in which the timescale of the project is partitioned into timeboxes, each corresponding to an iterative cycle. Within each timebox, requirements are prioritised, e.g. into essential, important, desirable and deferred. All essential requirements must be delivered, that is, incorporated into the emerging prototype system at the end of the timebox, while as many of the important and desirable requirements as possible are delivered. Each timebox is relatively short, typically four to six weeks, and the project

then consists of a number of timeboxes in sequence. Such a development process is known as **rapid application development (RAD)**. This has had a significant impact upon software development practice, and takes a variety of different forms. Among the benefits claimed are that:

- ▶ there is a regular, predictable pattern of prototyping;
- ▶ regular assessment of the requirements helps keep track of the real priorities for the system;
- ▶ at the planned delivery date, the project manager is in a better position to make an intelligent choice between relinquishing functionality and delaying the delivery.

One significant form of RAD is the **Dynamic Systems Development Method (DSDM)**, devised by the not-for-profit DSDM Consortium.

DSDM

DSDM is a framework for rapid application development methods, driven by a number of principles including:

- ▶ decision making throughout involving all stakeholders, including users, clients and developers;
- ▶ frequent delivery of product;
- ▶ a focus on fitness for business purpose;
- ▶ incremental and iterative development and delivery;
- ▶ the integration of testing throughout the process.

The core techniques include timeboxing, but also:

- ▶ careful prioritisation of objectives;
- ▶ modelling of the software to explore the system from the users' perspective;
- ▶ prototyping to enable the users to check the detail of the requirements;
- ▶ systematic testing throughout the process, with a high proportion of testing done by non-technical users;
- ▶ configuration management.

DSDM is employed in the five phases of feasibility study, business study, functional model iteration, design and implementation.

A DSDM method can also be described as an agile method, to which we turn next.

SAQ 13

What is the main feature that distinguishes a RAD method from any other iterative method?

ANSWER.....

The main distinguishing feature is timeboxing.

5.3 Agile development

Documenting the development of a system – such as by creating and recording models of the evolving system – is usually considered to be an integral part of a systematic engineering approach to software development.

SAQ 14

State some of the benefits of documenting the development of a system.

ANSWER.....

As noted in *Unit 1*, Subsection 6.3, documenting the development of a system can be used:

- ▶ to help facilitate communication between teams of developers;
 - ▶ retrospectively, when the system is in operation, to help those maintaining it understand its structure;
 - ▶ to aid future reuse of the system.
-

Despite the benefits, having to follow the rules of a particular software development method and keep documentary records of development has proved unpopular with some software developers, who perceive that this distracts them from their main pursuit of designing and implementing software. Such record keeping adds cost and so is not always popular with project managers either. Managers also wish to maintain control of their projects, but not to over-control and alienate the software developers, who are their most valuable resource.

The response to such issues has been the emergence of **agile methods**. People using these approaches find great merit in the informal approaches of small-team, small-company software development, that is, where a small team works on an ambitious project, operating outside normal company policies and to a tight schedule. Such 'uncontrolled' development processes often result in high-quality software at low cost, and have been employed for a long time, though often not openly, and how they succeeded was not widely understood. Agile methods make this kind of development explicit and visible, and thus acceptable to managers and stakeholders.

In 2001, a number of leaders within the object-oriented movement and others interested in iterative methods met to discuss the bureaucracy of current methods and how to give freer rein to the creative energy of software developers. They established the Agile Alliance from which emerged agile methods which have the following features.

- ▶ They do not include prescriptions concerning what should be done when; they do not plan in the conventional sense.
- ▶ They are documentation-light, which is not to say that no modelling is done, but rather that models (usually UML models) are developed quickly to explore complex alternative designs, and are then discarded instead of being recorded as project documentation.
- ▶ The code is implemented by the person who designed it, rather than being handed from designer to programmer to implement.

There are a number of agile methods, perhaps the best known of which is **eXtreme Programming (XP)**, which was mentioned in *Unit 1*, Subsection 5.3. The guiding principles of XP can be set out as follows.

- ▶ Each software release is a small increment on the previous one, scoped by a combination of business priorities and technical realities; this scope can be changed if things do not turn out as expected.
- ▶ Design is kept as simple as possible, with frequent refactoring.
- ▶ Tests are developed by programmers and clients, and tests must be passed in order to proceed: the development is test driven.
- ▶ Coding is done as pair programming, with two programmers sharing a single computer and producing code to standards, which enhances communication with other programmers.

- ▶ Each element of new code is immediately integrated into the overall system.
- ▶ The team includes a client; all code is owned collectively and can be changed by anybody.
- ▶ People only work the normal 40-hour week.

SAQ 15

State some features that distinguish an agile method from iterative methods in general.

ANSWER.....

Three main distinguishing features were mentioned above.

- 1 Agile methods do not involve the amount of planning involved in most iterative methods.
- 2 Agile methods are documentation-light.
- 3 In an agile method the code is implemented by the person who designed it, rather than being handed from designer to programmer.

SAQ 16

Identify one aspect which RUP, RAD and agile methods all have in common.

ANSWER.....

They all adopt an iterative approach.

5.4 Open source development

You have learnt that agile methods reject some of the conventional wisdom of software engineering, such as the need to plan development explicitly. However, some people, for some kinds of project, reject even more of the framework of software engineering good practice. They have been derogatorily referred to as 'hackers', but the work of these people has often been successful, and what has evolved is a new approach to developing software, now commonly known as **open source software development**.

The key feature of **open source software** is that all the source code is made publicly available on a website, free for anybody to copy, modify and use, subject to the licence agreements attached to the source, which effectively make the source code into public goods. This means that if you have a use for the software, you can take it and use it. If you want to make changes and adaptations for your own use, then you are also free to do that.

A consequence of making software open source is that it becomes open to improvement by anybody who has an interest in improving it. Users of the software who are also sufficiently expert to fix defects or add new functions can do so, for no payment. Contributors may benefit in various ways, such as using the improved software themselves, personal development, learning from other very able people working on the software, recognition from peers and perhaps being able to sell their expertise about the software. Open source software has achieved enormous popular appeal, because its development processes have the reputation of delivering high-quality software at relatively low cost.

Eric Raymond in *The Cathedral and the Bazaar* (1999) described how the open source approach to developing software came to prominence in 1991 when a student, Linus Torvalds, sought assistance with the development of an operating system. Others helped out, modifying and improving his code to produce a stable and competitive

system – Linux – and thus the open source movement began. This was made possible because of support by the Free Software Foundation, a not-for-profit organisation founded in 1985, which had created a culture of voluntary contributions, and because the establishment of the Web enabled easy communication between dispersed people.

Set in the context of the established formal software development methods of the early 1990s, characterised graphically by Raymond as ‘cathedrals’, open source software development was radically new, even revolutionary, and its informality warranted Raymond’s term, the ‘bazaar’. This new movement was largely overlooked until products like Linux began to find favour, and exhibit quality and stability – attributes so often absent in proprietary software. How did this come about without tight managerial controls and prescribed methods and procedures?

The key ingredients of open source software development are worth examining.

- ▶ Open code – developers other than the person who developed the code can access it freely to propose changes at any time.
- ▶ Established and proven architecture that is highly modular – everyone understands what the overall system being developed is, and how the parts fit together, so that individuals can work on components independently of others working on the software.
- ▶ Parallel development – the independent components enable development and updates to proceed in parallel, unaffected by each other, which enables very rapid development of the whole system.
- ▶ Duplicate development – many people can be working on the same component, with the best solution being the one chosen.
- ▶ Much of the software is developed directly in code.
- ▶ Rapid peer comment and feedback when a new or updated component is proposed for inclusion, with the added advantages of subsequent rounds of discussion from the original contributor and reviewers.
- ▶ Highly talented and highly motivated developers are involved.
- ▶ Acknowledged leaders and experts may be party to the review processes, and may often be the ultimate arbiters on whether a contribution gets accepted.
- ▶ Increased user involvement because open source software is often software that developers themselves use.
- ▶ Rapid cycle of releases – the complete system may be rebuilt daily, or even more frequently, to ensure that the latest improvements are available to all, and that new contributions are given the ultimate test of full use in context.
- ▶ A very loose ‘management’, though this may vary depending upon the particular lead person and the other people involved.
- ▶ Support tools that facilitate communication and the management of software versions.

These points are based on Feller and Fitzgerald’s book, *Understanding Open Source Software Development*.

Open source development is often characterised as being done using voluntary labour, but this is not always the case. Some companies now pay their developers to work on open source projects subject to the controls of open source rather than the controls of the company. NetBeans was (and is still being) developed in just this way, led by Sun Microsystems.

On the negative side, open source, seen as a software development process to be managed, is risky, not least because it is generally not possible to schedule the work to ensure that development reaches predefined stages at predictable times.

SAQ 17

State some key features that distinguish open source development from the iterative methods of development that we have previously discussed.

ANSWER.....

Here are three distinguishing features arising from the discussion above.

- 1 Anyone can join in – there is no project team as such.
- 2 There is no defined project period, and no defined deployment stage – the software continues to be improved for as long as people are interested in doing so.
- 3 There is no defined project method – contributors can work on parts of the evolving software however they see fit.

In this final section of the course you have learnt about four distinct ways of developing software: RUP, RAD and agile methods, and open source development.

6

Summary

The aim of this unit was to extend your appreciation of how complex systems are developed, and of how professional software development is carried out, by introducing you to a range of topics that are significant in these contexts.

We considered at two levels of detail how a complex system might be designed: first, at the level of the system's logical architecture, guiding the design of its overall 'shape'; and second, at the level of design patterns, guiding the design at a more detailed level, considering how objects are created, for instance.

Complex systems are often used by several people simultaneously, and we discussed some particular issues relevant in this context, including concurrency and distribution.

Finally, the unit concluded by introducing you to some relatively new ways of developing software, which you are very likely to encounter again if you pursue study in this area, or indeed if you work as a professional software developer. You have, we hope, realised through this that the knowledge and skills you have been developing in M256 can be applied in a wide range of software development contexts.

LEARNING OUTCOMES

After studying this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ reason about, and illustrate in a package diagram, a system's logical architecture;
- ▶ explain and apply the Façade pattern;
- ▶ identify problems caused by multiple coordinating objects;
- ▶ explain and apply the Singleton pattern;
- ▶ explain the Publish–Subscribe pattern;
- ▶ reason about a system's deployment;
- ▶ understand the key issues that can arise in multi-user systems, and how they may be addressed;
- ▶ outline the key features of each of the following methods of development: RUP, RAD, agile and open source.

Glossary

agile method An approach to software development typically involving a small team working on an ambitious project, operating outside normal company policies and to a tight schedule.

architecture A term used in different ways in software engineering, the common theme being concentration on the overall organisation of a system: viewing the system in some abstract, high-level way, ignoring smaller details.

back end See storage tier.

business domain tier In a tiered architecture, a tier corresponding to the core system. This contains the parts of the system specific to a business area, which process data and generate information.

client-server architecture A logical architecture consisting of two tiers: a client and a server.

computer architecture A view of the different computer elements that a system uses – hardware elements such as different processors and so on.

concurrency control Regulating access to shared data to prevent problems associated with concurrent access.

concurrent access (to a system) Simultaneous use of a system by two or more users or clients.

dependency There is a dependency, or usage, between two components when elements in one component rely on, or in some way use, elements in the other.

deployment How the various components that go to make up a distributed system are physically arranged, i.e. what physical computer or computers each part of the software runs on, and how these computers communicate.

deployment diagram A diagram illustrating how a system's components are deployed.

design pattern A tried and tested outline design applicable across a range of software systems, describing a recurring theme in elegant and respected designs. Typically, the description takes the form of a 'solution' which can be applied to a 'problem' which may arise in different contexts. Design patterns are commonly classified as *structural* (concerned with the system's structure), *creational* (concerned with how objects are created), or *behavioural* (concerned with interactions within the system).

distributed system A system involving more than one computer connected by a network.

Dynamic Systems Development Method (DSDM) A significant form of RAD devised by the not-for-profit organisation DSDM Consortium.

eXtreme Programming (XP) An example of an agile method.

façade class A class added to a component, as described by the Façade pattern, to provide a primary means of interaction with the component. In M256, such a class is termed the *coordinating class*.

façade object An instance of a façade class. In M256, such an object is termed the *coordinating object*.

Façade pattern An example of a structural design pattern.

front end See user interface tier.

information system A system which involves calculating and providing information for the user.

layer See tier.

lock A means of providing uninterrupted access to an object.

logical architecture A view of a system's software which is concerned with the organisation of the software classes into larger components, and which concentrates on how the overall system is constructed from major components, how the components are connected and how they work together.

model–view separation principle A software design principle which states not only that the business domain (the 'model') and the user interface (the 'view') should be separate entities, but that the business domain should not depend on the user interface.

multi-user system A system which several users may access via different computers.

network architecture A view of how the different computers involved in a system are connected.

open source software Software developed using open source software development.

open source software development An approach to software development whose key feature is that source code is made publicly available on a website, free for anybody to copy, modify and use (subject to licence agreements).

package diagram A diagram illustrating a system's components and their dependencies.

Publish–Subscribe pattern An example of a behavioural design pattern.

rapid application development (RAD) A software development method based on timeboxing.

Rational Unified Process (RUP) A description of a set of iterative software development methods having in common a set of 'best practice' elements. The RUP was devised by the software development experts who were also involved in developing the UML.

Remote Method Invocation (RMI) A Java mechanism that allows a program to send messages to a Java object which is in a different JVM and running on a different computer.

serialisable (object) A serialisable object is one whose class implements the `Serializable` interface and can therefore be represented in a way that enables it to be stored in a file (i.e. persist) or sent over a network.

singleton An object that is the only instance of its class, as described by the Singleton pattern.

Singleton pattern An example of a creational design pattern.

storage tier In a tiered architecture, a tier which provides a persistent data-storage mechanism, such as a database. Also known as the back end of the system.

three-layered architecture See three-tiered architecture.

three-tiered architecture A logical architecture consisting of three tiers: a user interface tier, a business domain tier and a storage tier. Also known as a three-layered architecture.

tier An element in a system's logical architecture. Also known as a layer. A tier can be considered as a component with distinct responsibilities such that:

- ▶ a lower tier has lower-level responsibilities;
 - ▶ higher tiers have responsibilities that are increasingly specific to the business area;
 - ▶ higher tiers may depend on lower tiers but not vice versa.
-

tiered architecture A logical architecture which is structured as a series of tiers.

timeboxing The practice of partitioning the timescale of a software development project into timeboxes, each corresponding to an iterative cycle and in each of which requirements are prioritised. Timeboxing is a key feature of RAD.

usage See dependency.

user interface tier In a tiered architecture, a tier giving a 'view' of the information provided by the system, using windows, spreadsheets, forms etc. Also known as the front end of the system.

References

Feller, J. and Fitzgerald, B. (2002) *Understanding Open Source Software Development*, Addison-Wesley.

Gamma, G., Helm, R., Johnson, R. and Vlissides, J. (1995) *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional.

The Hillside Group, <http://hillside.net> (Accessed 12 March 2007).

Rational Software (1998) *Rational Unified Process. Best Practices for Software Development Teams* [online], http://www-128.ibm.com/developerworks/rational/library/content/03July/1000/1251/1251_bestpractices_TP026B.pdf (Accessed 13 March 2007).

Raymond, E. (1999) *The Cathedral and the Bazaar: Musing on Linux and Open Source by an Accidental Revolutionary*, O'Reilly.

Index

- A
 - Agile Alliance 43
 - agile method 43–44
 - architecture 7
- B
 - back end 10
 - behavioural design pattern 14
 - business domain tier 10
- C
 - client–server architecture 10
 - component 7, 29, 40
 - computer architecture 7
 - concurrency control 36
 - concurrent access 11, 28
 - coordinating class 17
 - coordinating object 17
 - creational design pattern 14
- D
 - dependency 8
 - deployment 28
 - deployment diagram 29
 - design pattern 13
 - design principle 13
 - distributed system 28
 - documentation 43
 - Dynamic Systems Development Method (DSDM) 42
- E
 - eXtreme Programming (XP) 43
- F
 - façade
 - class 14
 - object 15
 - Façade pattern 14
 - Free Software Foundation 45
 - front end 10
- I
 - iterative method 39, 44
- L
 - layer 9
 - legacy system 17
 - Linux 45
 - locking 37
 - logical architecture 7, 40
- M
 - model–view separation principle 11
 - multi-user system 28
- N
 - NetBeans 45
 - network architecture 7
- O
 - object-oriented software component 7
 - open source software 44
 - open source software development 44
- P
 - package diagram 7
 - pair programming 43
 - persistence 28, 30
 - prototype 39
 - prototyping 42
 - Publish–Subscribe pattern 14, 25, 38
- R
 - rapid application development (RAD) 42
 - Rational Software Corporation 40
 - Rational Unified Process (RUP) 40
 - refactoring 43
 - Remote Method Invocation (RMI) 30
 - RemoteException 31
- S
 - scenario 40
 - serialisable (object) 30
 - singleton 22
 - Singleton pattern 14, 17, 22
 - software quality 40
 - storage tier 10
 - structural design pattern 14
- T
 - three-layered architecture 10
 - three-tiered architecture 10
 - tier 9
 - tiered architecture 9
 - timeboxing 41–42
 - Torvalds, Linus 44
- U
 - UML 40
 - usage 8
 - use case 40
 - user interface tier 10
- W
 - waterfall method 39
- X
 - XP (eXtreme Programming) 43